

Efficient Diffusion Models Fine-Tuning for Microscopy Imaging

Biel Castaño Segade

M2 Mathématiques, Vision, Apprentissage - École Normale Supérieure Paris-Saclay

Internship at the Biology Institute of École Normale Supérieure - PSL

Supervised by Anis Bourou and Thomas Boyer



Acknowledgements

I would like to express my heartfelt gratitude to everyone who made this internship possible. First and foremost, I am deeply thankful to Anis Bourou and Thomas Boyer for selecting me for this opportunity and for supervising my work, as well as to Auguste Genovesio for accepting me into his laboratory as an intern and guiding me throughout my time there. I am also sincerely appreciative of Bruno Galerne for kindly agreeing to serve as the scientific referee for this internship.

Contents

1	Introduction	3
1.1	Context	3
1.2	Motivations	3
1.3	Internship Objectives	4
1.4	Internship Timeline	4
2	Background and Existing Approaches	5
2.1	Denoising Diffusion Probabilistic Model (DDPM)	5
2.2	Denoising Diffusion Implicit Model (DDIM)	8
2.3	Diffusion Model as a Stochastic Differential Equation	9
2.4	Latent Diffusion Models	10
2.5	Fine-Tuning Deep Learning Models	12
3	Implementation	16
3.1	Stable Diffusion Components	16
3.2	Denoiser Architecture	16
3.3	Training script	18
3.4	Image-to-image Pipeline for Image Translation	18
4	Experiments	19
4.1	Datasets	19
4.2	Validation metrics: FID	19
4.3	Testing Generalization Capabilities	21
4.4	Conducted Experiments	22
5	Results	23
5.1	From Scratch Method	23
5.2	FID Convergence	23
5.3	Image Quality Analysis	25
5.4	Generalization Capabilities	27
5.5	Images Evolution During Training	29
5.6	Image-to-Image Translation	29
5.7	Organoids Images Generation	30
6	Conclusion	31
A	Appendix	36

1 Introduction

This report summarizes a six-month internship conducted at the Institut de Biologie de l'École Normale Supérieure as part of the Master 2 program in Mathematics, Vision, and Learning at ENS Paris-Saclay. The internship was completed in the Computational Bioimaging and Bioinformatics lab, led by Dr. Auguste Genovesio, with guidance from Anis Bourou and Thomas Boyer. The primary focus of the work was on the fine-tuning of conditional diffusion models and their applications in image-to-image translation, specifically for biological images. The implementation of this project is available on GitHub.

1.1 Context

Deep learning, a subset of machine learning, is based on deep, multilayered neural networks. These techniques are highly effective in learning data distributions across various domains, making them valuable in numerous fields, such as robotics, computer vision, and natural language processing.

In the field of computer vision, deep learning has significant applications in microscopy imaging such as phenotype detection [1]. Before the advent of deep learning, microscopy imaging depended on conventional image processing methods such as thresholding and edge detection to examine cellular structures [2, 3]. Although these methods were useful, they often struggled with the complexity and variability of biological imaging data. The emergence of deep-learning generative approaches has transformed the analysis of microscopy images. For example, Conditional GANs have been used to uncover hidden cell phenotypes, as demonstrated in [4]. However, while addressing some of the limitations of traditional techniques, these approaches introduced new challenges, such as training instability and model collapse [5].

To address these challenges, diffusion models have quickly emerged as the state-of-the-art in generative modeling [6, 7]. These models have opened up a wide range of applications in microscopy imaging, including phenotype detection and high-quality image-to-image translation [1]. However, diffusion models require extensive data and computational resources, which can limit their application in data-scarce environments. Fine-tuning pre-trained models has proven to be a successful approach to this problem across different generative models [8, 9]. However, simple fine-tuning of diffusion models often results in overfitting and catastrophic forgetting, even when the source and target domains are closely related [10, 11]. Recent progress has introduced more effective techniques for fine-tuning generative models [12–14]. The goal of the internship is to apply these techniques to image-to-image translation and assess their effectiveness in identifying subtle phenotypes in microscopy images.

1.2 Motivations

Image-to-image translation using deep learning techniques, such as diffusion models, offers significant benefits for biomedical imaging. These methods have the potential to uncover subtle variations in cellular and tissue structures that may not be immediately apparent through traditional imaging analysis. By generating high-quality translated images, these models can assist researchers in visualizing and interpreting complex biological data, which can lead to a better understanding of various diseases and conditions.

The internship focuses on exploring and adapting fine-tuning methods for diffusion models to enhance their generative capabilities with limited data, which, in turn, will improve their performance in image-to-image translation tasks. These models will be applied to different microscopy datasets to achieve specific research objectives:

- **Cancer Research with the BBBC021 Dataset:** The BBBC021 dataset [15] contains high-content imaging data of cells under different conditions, such as treatment with DMSO or Latrunculin, which are often used in cancer research. This dataset can be useful for cancer studies because it allows researchers to analyze cellular responses to treatments in detail. By applying image-to-image translation to the BBBC01 dataset, researchers can better detect changes in cells, helping to identify new biomarkers and understand cancer progression and treatment effects.
- **Neuro-Developmental Disorders (NDDs):** Neuro-developmental disorders affect approximately 10% of children causing lifelong disabilities [16]. There is a critical need for new therapeutic strategies to address these disorders. Deep learning has already advanced the study of NDDs, as recent research shows [2, 4].

At the Valérie Mezger lab, various neural models have been imaged to capture the intricate aspects of these disorders. Fine-tuning diffusion models on these images can help detect subtle phenotypic variations, which may provide deeper insights into the pathophysiology of NDDs and guide the development of targeted therapies.

By successfully adapting and applying these fine-tuning methods to different datasets, we aim to improve the generative capabilities of diffusion models, which will concurrently enhance their effectiveness in image-to-image translation for biomedical research. Ultimately, this could contribute to a better understanding of both neuro-developmental disorders and cancer, and support the development of innovative therapeutic strategies.

1.3 Internship Objectives

The main objectives of this internship are outlined as follows:

1. **Understand the Foundations of Diffusion Models and Fine-tuning techniques:** Review the theoretical foundations of diffusion models and study existing methods for fine-tuning them.
2. **Fine-tuning a Conditional Diffusion Model:** Fine-tune a conditional diffusion model that can be initialized using publicly available pre-trained weights, such as those from Stable Diffusion. The model should be fine-tuned using one of the selected fine-tuning methods. Specifically, this internship focuses on the following methods: SVDiff, LoRA, full fine-tuning, and attention fine-tuning, as detailed in Section 3.2.
3. **Evaluate Fine-Tuning Methods:** Conduct both quantitative and qualitative evaluations to determine which fine-tuning method most effectively adapts the model's underlying distribution from natural images to microscopy images. This involves selecting appropriate metrics and visualization techniques to assess the performance of each method.
4. **Develop an Image-to-Image Translation Pipeline:** Create a translation pipeline capable of performing image-to-image translation using models fine-tuned with the methods mentioned above.

1.4 Internship Timeline

The internship spanned 6 months, or 23 weeks, excluding vacation periods. The following outlines the timeline of the internship:

- **Weeks 1 to 4:** Focused on gaining foundational knowledge of diffusion models, as this was a new area for me. This included studying relevant literature and completing online courses.
- **Weeks 5 and 6:** Reviewed the literature on fine-tuning methods and reproduced the SVDiff code on an unconditional model using only a single class from the DMSO dataset. The goal was to verify the model's ability to shift the distribution towards biological images (from the DMSO dataset), generating images at a resolution of 512x512.
- **Weeks 7 and 8:** Modified the code and searched for new pre-trained checkpoints to enable the model to generate images at a resolution of 128x128. Set up the environment for parallel training and configured results to be uploaded to Weights & Biases (wandb).
- **Week 9:** Incorporated additional fine-tuning methods, including LoRA, full fine-tuning, and attention-based fine-tuning. Also, implemented a validation process using the FID score.
- **Weeks 10:** Conducted a literature review on conditional image generation using diffusion models. Developed a conditional model that directly accepts a binary label as input, eliminating the need for text input, a tokenizer, and a text encoder.
- **Week 11 and 12:** Designed an image-to-image translation pipeline using the trained conditional model, capable of taking an input image from one class and translating it into another class.
- **Weeks 13 to 15:** Developed and implemented a validation process to evaluate the generalization capabilities of each model during image generation.

- **Week 16:** Prepared the environment and adapted the code to run on the Jean Zay supercomputer, enabling rapid execution of multiple experiments.
- **Week 17:** Defined and initiated a series of experiments to assess how each fine-tuning method performs with different training dataset sizes.
- **Weeks 18 and 19:** Writing of the internship report.
- **Week 20:** Prepared the presentation and tried to replicate experiments using a second more complex dataset.
- **Weeks 21 to 23:** Conducted additional experiments and started drafting an article for a workshop based on the project.

2 Background and Existing Approaches

In this chapter, we explore Denoising Diffusion Probabilistic Models (DDPMs) and Denoising Diffusion Implicit Models (DDIMs) from both conceptual and practical viewpoints. We also examine Latent Diffusion Models and discuss existing fine-tuning methods applicable to these models. Lastly, we review a method for performing image-to-image translation using diffusion models.

2.1 Denoising Diffusion Probabilistic Model (DDPM)

In this subsection, we will discuss the first type of diffusion models that were designed, known as **Denoising Diffusion Probabilistic Models (DDPMs)** [17, 18]. It is important to note that the concepts explained here may differ for other types of diffusion models.

Diffusion models are a type of generative model that address the challenge of producing high-fidelity and diverse samples. The central idea is to iteratively refine a corrupted sample, under the premise that reconstructing a sample from a slightly degraded state is more feasible than from a heavily corrupted one. These models start with random noise—typically Gaussian—and gradually reduce this noise through a sequence of steps to produce a realistic output.

The training of diffusion models involves two main stages. In the **forward process**, each state x_k of the data is slightly more corrupted than the previous state x_{k-1} by adding Gaussian noise. This can be represented as:

$$q(x_k|x_{k-1}) = \mathcal{N}(\sqrt{1 - \beta_k} x_{k-1}, \beta_k I_d), \quad (1)$$

where x_0 is the original data, (β_k) are predefined noise scales, and I_d is the identity matrix. Alternatively, this can be written as:

$$x_k = \sqrt{1 - \beta_k} x_{k-1} + \sqrt{\beta_k} \epsilon_k, \quad \epsilon_k \sim \mathcal{N}(0, I_d). \quad (2)$$

After many iterations, this forward process results in x_T , which is nearly pure Gaussian noise. The distribution at any step t can be computed recursively as:

$$\begin{aligned}
x_t &= \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_{t-1} \\
&= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2} + \sqrt{1 - \alpha_t}\epsilon_{t-1} \\
&= \dots \\
&= \sqrt{\prod_{i=1}^t \alpha_i}x_0 + \sqrt{1 - \prod_{i=1}^t \alpha_i}\epsilon_0 \\
&= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_0 \\
&\sim \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I_d),
\end{aligned}$$

where $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$. Thus, the distribution of x_t given x_0 can be written as:

$$q(x_t | x_0) = \mathcal{N}(x_t | \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I_d). \quad (3)$$

This expression is particularly useful during the training of the diffusion model, as it allows us to sample any noisy state x_t directly from the initial state x_0 . By having a closed-form expression for $q(x_t | x_0)$, we can efficiently compute the distribution at any timestep t without needing to perform recursive calculations for each intermediate state. This not only simplifies the implementation but also significantly reduces the computational cost, making the training process much more efficient.

The **backward process** aims to reconstruct the original data by reversing the forward diffusion, gradually removing noise to recover the data sample from a noisy input. Given that the reversal of a diffusion process is also a diffusion process, and assuming small enough noise increments, the reverse process is modeled with the same functional form:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)), \quad (4)$$

where $\mu_\theta(x_t, t)$ and $\Sigma_\theta(x_t, t)$ represent the mean and variance estimated by a neural network with learnable parameters θ .

Thus, the training of diffusion models focuses on learning the parameters μ_θ and Σ_θ of the reverse process $p_\theta(x_{t-1} | x_t)$. The objective is to maximize the model's likelihood of the data, $p_\theta(x_0)$, representing the probability of the true data being generated by the model. This likelihood can be written as the probability over all possible trajectories from the noisy state x_T back to the clean state x_0 . We can equivalently maximize the log-likelihood, given by:

$$L = \mathbb{E}_{x_0 \sim \mathbb{P}_{\text{data}}} [\log \mathbb{P}_\theta(x_0)], \quad (5)$$

where $\mathbb{P}_\theta(x_0)$ is the probability assigned to a true data point x_0 by the learned reverse model.

To optimize the model, we aim to align the backward process with the forward diffusion process. When the steps in these processes are sufficiently small, their distributions should become similar, allowing us to formulate an objective function analogous to the Variational Autoencoder (VAE) framework [19, 20]. The Evidence Lower Bound (ELBO) for this model can then be derived as follows:

$$\begin{aligned}
L &\geq \text{ELBO}_\theta(\mathbf{x}) \\
&= - \sum_{t=1}^T \mathbb{E}_{q(\mathbf{x}_t | \mathbf{x}_0)} \left[\frac{1}{2\sigma_q^2(t)} \cdot \frac{(1 - \alpha_t)^2 \bar{\alpha}_{t-1}}{(1 - \bar{\alpha}_t)^2} \|\hat{\mathbf{x}}_\theta(\mathbf{x}_t) - \mathbf{x}_0\|^2 \right].
\end{aligned} \quad (6)$$

where $\hat{x}_\theta(x_t)$ is the prediction of the initial state generated by a neural network, given a state x_t that has been corrupted by noise. Many sources online explain how to derive this ELBO [21, 22], and we avoid the details of

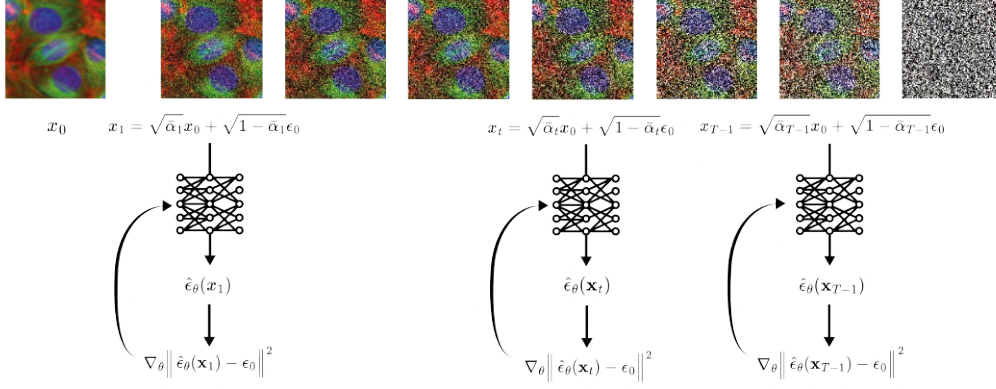


Figure 1: Diagram of the training procedure for a DDPM, using an image from the BBBC021 dataset.

the derivation in this report. However, it should be noted as an indication that the derivation is based on the definition of expectation, Jensen’s inequality, and Bayes’ theorem.

We can find an ELBO equivalent to Equation 6 but using a neural network $\hat{\epsilon}_\theta$ that predicts the noise ϵ_0 that has been added to obtain x_t [21]. In this case, the ELBO would be

$$\text{ELBO}_\theta(\mathbf{x}) = - \sum_{t=1}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[\frac{1}{2\sigma_q^2(t)} \cdot \frac{(1-\alpha_t)^2 \bar{\alpha}_{t-1}}{(1-\bar{\alpha}_t)^2} \|\hat{\epsilon}_\theta(\mathbf{x}_t) - \epsilon_0\|^2 \right]. \quad (7)$$

where

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon_0$$

Our goal is to train a neural network ϵ_θ so that its predicted noise $\hat{\epsilon}_\theta$ closely matches the actual noise ϵ_0 . The following training procedure can be used to achieve this [21], as illustrated in Fig. 1 :

Algorithm 1 Training Algorithm for Noise Prediction Network $\bar{\epsilon}_\theta$

for each image x_0 in the training dataset **do**

repeat

- Randomly sample a time step $t \sim \text{Uniform}[1, T]$
- Generate a sample $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon_0$, where $\epsilon_0 \sim \mathcal{N}(0, I_d)$
- Perform a gradient descent step on $\nabla_\theta \|\hat{\epsilon}_\theta(x_t) - \epsilon_0\|^2$

until convergence

end for

To generate images after the model is trained, we begin by sampling Gaussian noise and then progressively denoise it using the trained model. For each time step $t \in \{1, \dots, T\}$, we need to iteratively sample x_{t-1} given x_t . We can find that the distribution $p_\theta(x_{t-1} | x_t)$ can be derived as follows [21]:

$$\begin{aligned} \mathbf{x}_{t-1} &\sim p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) \\ &= \mathcal{N}(\mathbf{x}_{t-1} | \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \hat{\epsilon}_\theta(\mathbf{x}_t) \right), \frac{(1-\alpha_t)(1-\sqrt{\bar{\alpha}_{t-1}})}{1-\bar{\alpha}_t} \mathbf{I}) \\ &= \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \hat{\epsilon}_\theta(\mathbf{x}_t) \right) + \frac{(1-\alpha_t)(1-\sqrt{\bar{\alpha}_{t-1}})}{1-\bar{\alpha}_t} \mathbf{z} \end{aligned} \quad (8)$$

where $z \sim \mathcal{N}(0, I_d)$. Notice in Equation 8 that the first term already provides an estimate of x_0 . This might lead one to question why we need to proceed step-by-step from x_T to x_0 if we can directly obtain this estimate of x_0 . The reason is that the quality of this estimate improves as t decreases. If we attempt to predict x_0 directly from x_T , the estimate would be highly inaccurate. However, by progressively following the denoising process, each

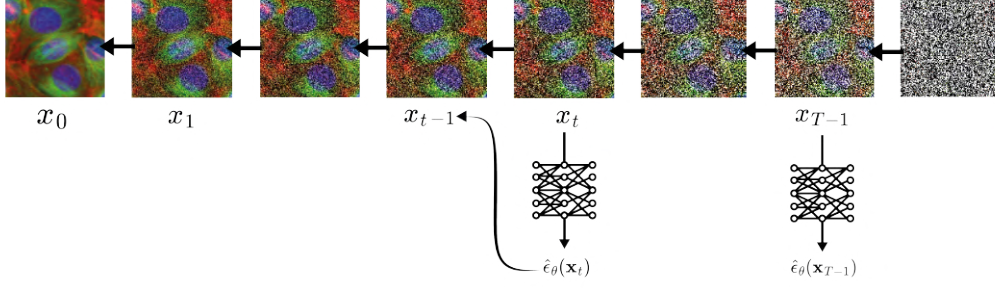


Figure 2: Diagram of the inference procedure for a DDPM, using an image from the BBBC021 dataset.

step yields increasingly precise estimates, ultimately leading to a more accurate prediction of x_0 when derived from x_1 . Therefore, we can use the following algorithm to generate images [21], as illustrated in Fig. 2:

Algorithm 2 Inference Algorithm for Noise Prediction Network $\bar{\epsilon}_\theta$

- 1: • Sample a white noise vector $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
 - 2: **for** $t = T, T - 1, \dots, 1$ **do**
 - 3: • Calculate $\hat{\epsilon}_\theta(\mathbf{x}_t)$ using the trained model.
 - 4: • Compute x_{t-1} according to equation 8.
 - 5: **end for**
-

It is important to note that a major drawback of DDPMs is that each step requires passing through the entire network, which can be slow for a typical choice like $T = 1000$. While it is possible to reduce the number of steps, this significantly worsens the results.

2.2 Denoising Diffusion Implicit Model (DDIM)

A solution to speed up diffusion models was presented in [23]. This paper introduced the Denoising Diffusion Implicit Model (DDIM). The principal insight of their work is that the DDPM objective depends only on the marginals $q(x_t | x_0)$, and not directly on the joint distribution $q(x_{1:T} | x_0)$. Given that multiple forward distributions can share the same marginals, they explored alternative forward processes that are non-Markovian. To define such an approach, they proposed a forward process that conditions on both the initial state and the previous state. With this in mind, they propose the following joint distribution:

$$q(x_{t-1} | x_t, x_0) = \mathcal{N} \left(\sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 I_d \right) \quad (9)$$

where $\sigma \in \mathbb{R}_{\geq 0}^T$ is a real vector, and from which we can deduce the forward process using Bayes' rule:

$$q_\sigma(x_t | x_{t-1}, x_0) = \frac{q_\sigma(x_{t-1} | x_t, x_0) q_\sigma(x_t | x_0)}{q_\sigma(x_{t-1} | x_0)} \quad (10)$$

This forward process is no longer Markovian since each x_t can depend on both x_{t-1} and x_0 . The magnitude of σ controls the level of stochasticity in the process. If $\sigma = 0$, the process becomes deterministic since x_{t-1} is fixed for a given x_0 and x_t . On the other hand, if

$$\sigma_t = \sqrt{\frac{1 - \alpha_{t-1}}{1 - \alpha_t}} \sqrt{\frac{1 - \alpha_t}{\alpha_{t-1}}} \quad (11)$$

for $t \in \{1, \dots, T\}$ the process becomes a DDPM. For simplification, we will assume throughout the rest of the report that a DDIM is deterministic.

The corresponding reverse process is:

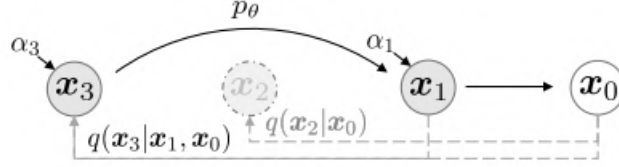


Figure 3: Graphical model for accelerated generation presented in [23], where $\tau = [1, 3]$.

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \frac{x_t - \sqrt{\bar{\alpha}_t}\hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 \mathbf{I}\right) \quad (12)$$

As demonstrated in [23], the training objective of DDPM can also be applied to DDIM, independent of the choice of σ . This implies that there is no practical difference between DDPM and DDIM during training; the significant differences occur during sampling.

To generate a sample of x_{t-1} given x_t , we can apply the following equation:

$$x_{t-1} = \sqrt{\alpha_{t-1}} \left(\frac{x_t - \sqrt{1 - \alpha_t} \cdot \epsilon_\theta^{(t)}(x_t)}{\sqrt{\alpha_t}} \right) + \sqrt{1 - \alpha_{t-1} - \sigma_t^2} \cdot \epsilon_\theta^{(t)}(x_t) + \sigma_t \epsilon_t \quad (13)$$

where $\alpha_0 = 0$ and $\sigma_t = 0$.

Recall that the generative process in DDPMs requires sampling over T steps, matching the number of steps of the forward process. However, since the denoising objective defined in [23] depends only on the marginal distribution $q_\sigma(x_t | x_0)$ and not in the specific forward procedure, the generative process can be shortened to fewer steps. By choosing a subset of latent variables $\{x_{\tau_1}, \dots, x_{\tau_S}\}$ where τ is an increasing sequence within $[1, \dots, T]$, we can create a shorter trajectory that preserves the necessary marginals as illustrated in Fig. 3, resulting in significant computational savings. For instance, while DDPMs may require up to 1000 steps for sample generation, DDIMs can achieve similar results with as few as 100 steps.

Additionally, as we will see in the next section, we can invert the generative process to identify the specific Gaussian seed x_T that would produce a given image x_0 due to the deterministic nature of DDIMs.

2.3 Diffusion Model as a Stochastic Differential Equation

The diffusion process described in Section 2.1 can be extended to a continuous-time formulation. Instead of using the sequence of noise scales $\beta_{t=1}^T$ to define a sequence of distributions $q(x_t|x_{t-1})_{t=1}^T$, as in Equation 1, these distributions can be modeled through the evolution of a Stochastic Differential Equation (SDE).

To further build on this framework, we define a continuous-time diffusion process $x(t)$ [24], operating over the continuous interval $t \in [0, T]$. At $t = 0$, $x(0) \sim p_0$, where p_0 is the distribution derived from a dataset of independent and identically distributed samples. At $t = T$, $x(T) \sim p_T$, where p_T represents a distribution that can be easily sampled; in our case, a Gaussian distribution with fixed mean and variance. This diffusion process can be represented as the solution to an Itô stochastic differential equation:

$$dx = f(x, t)dt + g(t)dw \quad (14)$$

Where:

- w_t is a standard Wiener process ¹, also known as Brownian motion.

¹A Wiener process W_t is a continuous-time stochastic process that satisfies the following properties:

1. $W_0 = 0$,
2. W_t has independent increments,
3. for any $t > s$, the increment $W_t - W_s \sim \mathcal{N}(0, t - s)$ (i.e., normally distributed with mean 0 and variance $t - s$),
4. W_t has continuous paths almost surely.

- $f(\cdot, t) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a vector-valued function called the drift coefficient of $x(t)$, which controls the deterministic part of the process.
- $g(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$ is a scalar function known as the diffusion coefficient of $x(t)$, determining the magnitude of random noise.

By reversing the process, we can move from $x(T) \sim p_T$ back to $x(0) \sim p_0$. Notably, in [25] it was demonstrated that there exists a reverse diffusion process, evolving backward in time and governed by:

$$dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) d\bar{w} \quad (15)$$

where, $\bar{w}_t$ represents a Wiener process as time progresses in reverse from T to 0 and dt is an infinitesimal negative time increment. What we aim to learn is the score of each marginal distribution, $\nabla_x \log p_t(x)$, using a parametrized model. This would allow us to apply the reverse diffusion process from Equation 15 to generate samples from p_0 .

This perspective of modeling the diffusion process as an SDE is theoretical; in practice, we use diffusion models that are discretized versions of this concept, such as DDPMs or DDIMs. However, this approach is particularly relevant for this project in the context of DDIMs, since we can rewrite Equation 13 as:

$$\frac{x_{t-\Delta t}}{\sqrt{\alpha_{t-\Delta t}}} = \frac{x_t}{\sqrt{\alpha_t}} + \left(\sqrt{\frac{1 - \alpha_{t-\Delta t}}{\alpha_{t-\Delta t}}} - \sqrt{\frac{1 - \alpha_t}{\alpha_t}} \right) \epsilon_{\theta}^{(t)}(x_t) \quad (16)$$

From Equation 16 we can derive the corresponding Ordinary Differential Equation as [23]:

$$d\bar{x}(t) = \epsilon_{\theta}^{(t)} \left(\frac{\bar{x}(t)}{\sqrt{\sigma^2 + 1}} \right) d\sigma(t), \quad (17)$$

which can be treated as an Euler method over the given system, where the initial condition is $x(T) \sim \mathcal{N}(0, \sigma(T))$ for very large σ . This shows that with enough discretization steps, we can reverse the generation process, it is, predicting x_T for a given x_0 by simulating the reverse of Equation 17.

More specifically, as described in [6], the generative pass in DDIM is

$$x_{t-1} - x_t = \sqrt{\bar{\alpha}_{t-1}} \left[\left(\sqrt{\frac{1}{\bar{\alpha}_t}} - \sqrt{\frac{1}{\bar{\alpha}_{t-1}}} \right) x_t + \left(\sqrt{\frac{1}{\bar{\alpha}_{t-1}}} - 1 - \sqrt{\frac{1}{\bar{\alpha}_t}} - 1 \right) \epsilon_{\theta}(x_t) \right] \quad (18)$$

while in the limit of small steps the reverse of the generation process is

$$x_{t+1} - x_t = \sqrt{\bar{\alpha}_{t+1}} \left[\left(\sqrt{\frac{1}{\bar{\alpha}_t}} - \sqrt{\frac{1}{\bar{\alpha}_{t+1}}} \right) x_t + \left(\sqrt{\frac{1}{\bar{\alpha}_{t+1}}} - 1 - \sqrt{\frac{1}{\bar{\alpha}_t}} - 1 \right) \epsilon_{\theta}(x_t) \right] \quad (19)$$

The important point here is that in both Equations 18 and 19, we use the same parametrized neural network, ϵ_{θ} . This allows us to train the model solely for the generative process, and once trained, we can apply the reverse generative process using the same model. In other words, using the same model we can apply the inversion process in DDIMs to find a mapping from images x_0 back to their latent representations x_T , something that is not feasible with the stochastic nature of the DDPM forward process. This characteristic is crucial for tasks such as image translation, as we will discuss in Section 3.4.

2.4 Latent Diffusion Models

Diffusion models can become extremely computationally expensive when working with high-resolution images in pixel space. To address this, Latent Diffusion Models were presented in [26]. This kind of models rely on three key principles:

1. **Image Encoding:** The authors of this model use an autoencoder trained with a perceptual loss and a patch-based adversarial objective. This training ensures that the reconstructions produced by the decoder preserve local details and avoid blurriness. Specifically, given an image $x \in \mathbb{R}^{H \times W \times C}$, the encoder $\mathcal{E}$ produces a latent representation $z = \mathcal{E}(x)$, which is a downsampled representation of x . The decoder $\mathcal{D}$ then reconstructs the image as $\hat{x} = \mathcal{D}(z)$, aiming to make $\hat{x}$ as close as possible to the original x .
2. **Applying Diffusion Models to Latent Representations:** After training the autoencoder, a diffusion model is applied on the latent space generated by the autoencoder, rather than directly on the images. This latent space, which is lower-dimensional than the high-dimensional pixel space, is more suitable for likelihood-based generative models. This approach allows the model to emphasize the most meaningful semantic features of the data while also operating within a more computationally efficient, lower-dimensional framework. The diffusion model is implemented using a time-conditioned UNet [27], which will be detailed further in section 3.2.
3. **Conditioning by Cross-Attention:** The authors use a cross-attention mechanism to condition the diffusion model. Suppose we have the variable z_t representing the latent encoding of an image x at timestep t in the diffusion process. Let $\psi_i(z_t) \in \mathbb{R}^{N \times d_\epsilon^i}$ be the representation of z_t in an intermediate layer of the UNet that predicts noise $\hat{\epsilon}_\theta$. Additionally, suppose the image is paired with a class input y . An encoder τ_θ transforms this class into a representation $\tau_\theta(y) \in \mathbb{R}^{M \times d_t}$. These representations are then projected into three matrices Q , K , and V , known as query, key, and value, respectively, defined as follows:

$$Q = \psi_i(z_t)W_Q^i \quad (20)$$

$$K = \tau_\theta(y)W_K^i \quad (21)$$

$$V = \tau_\theta(y)W_V^i \quad (22)$$

where $W_Q^i \in \mathbb{R}^{d_\epsilon^i \times d}$ and $W_K^i, W_V^i \in \mathbb{R}^{d_t \times d}$ are learnable matrices for some $d \in \mathbb{N}$. Note that the conditioning data y can theoretically be any type of data, such as images or real values. However, in the implementation proposed in [26], the diffusion model is conditioned using tokenized text, making τ_θ a text encoder in this case.

The attention layer is then defined as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V. \quad (23)$$

This attention mechanism can be multi-headed [28], allowing the model to simultaneously focus on different representation subspaces at various positions. Notably, in their implementation, both the noise predictor ϵ_θ and the text encoder τ_θ are trained jointly.

This model has been widely adopted, with Stable Diffusion being one of the most well-known implementations. Stable Diffusion models are trained from scratch on very high-resolution images, large architectures, and massive datasets. Different versions of the model have varying sizes; for example, Stable Diffusion XL has 2.6 billion trainable parameters in the UNet, while Stable Diffusion 2.1 has 865 million. Both models have been trained over more than one billion optimization steps at different resolutions and on subsets of the LAION-5B dataset, which contains a vast collection of natural images with captions [29].

Although these trained models can generate high-quality images, fine-tuning is especially important because it allows us to retain the general knowledge learned from large datasets by a big model. This approach enables us to retrain a model using much smaller datasets or even with fewer computational resources. We will discuss this further in Section 2.5.

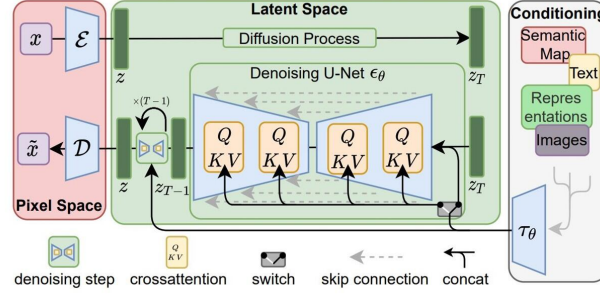


Figure 4: Stable Diffusion schema presented in [26]

2.5 Fine-Tuning Deep Learning Models

While Deep Learning has the potential to greatly improve biology research, there is the complication of the massive amount of data needed to train powerful Deep Learning systems from scratch. This challenge becomes even more difficult as deep learning technology advances because these models require enormous amounts of data for training, which is not always available. There are four main reasons why acquiring this much data is difficult in the biological domain:

- **Privacy restrictions:** Laws and regulations protect patient privacy, making it hard to collect large datasets.
- **Long-tailed distribution:** Many biological conditions are rare, so there's not a lot of data available for them.
- **Cost of data annotation:** Even when data is available, labeling it for Deep Learning training is expensive and time-consuming.
- **High cost of data generation:** Generating biological data often requires conducting costly experiments, which further limits the availability of large datasets.

To address data limitations, biological Deep Learning usually benefits from transfer learning [13]. This approach trains models in two steps:

- **Pre-training:** on large datasets.
- **Fine-tuning:** on smaller datasets for specific tasks.

This is especially helpful for powerful models that require massive amounts of training data. Diffusion models, in particular, often need extensive datasets to perform effectively. Historically, fine-tuning was primarily applied to discriminative models, but it has been adapted for use with generative models as well. In our case, since we have limited data and are planning to train a diffusion model on a specific downstream task, a fine-tuning approach would be highly beneficial.

We can classify fine-tuning methods into two main categories [13]:

- **Full Fine-Tuning:** This is the standard approach where all the parameters of a pre-trained model are adjusted during training for the new task. It requires a large amount of data and computational resources but can achieve high performance when sufficient data is available.
- **Parameter-Efficient Fine-Tuning (PEFT):** PEFT methods aim to reduce the number of parameters that need to be updated during training, making the process more efficient. This category can be further divided into:
 - **Adaptive Methods:** These methods introduce new, lightweight modules, often called "adapters," into the pre-trained model. These adapters allow the model to adapt to the new task without requiring all the original parameters to be fine-tuned. Examples include LoRA [30] and SVDiff [14], which will be explained in detail later.

- **Selective Fine-Tuning:** This approach involves fine-tuning only a specific subset of the pre-trained model’s parameters while leaving the remaining parameters frozen. For instance, fine-tuning only the parameters associated with bias terms [31] or normalization layers [32, 33].

Fine-tuning, especially in the context of medical image analysis, is essentially about balancing a model’s adaptation to specific tasks with the risk of overfitting. Although PEFT is widely adopted in other fields, its effectiveness for medical image analysis has not been comprehensively studied. One of the few surveys, if not the only one, that compares fine-tuning methods for medical imaging evaluates various state-of-the-art PEFT techniques across multiple medical datasets, covering both traditional and emerging tasks [13]. This survey aims to identify the most effective methods and set a benchmark for their application in medical vision tasks. Note that this paper is not dedicated to diffusion models, and is considering mainly CNNs and ViTs.

According to this survey, full fine-tuning does not always achieve the highest performance on downstream tasks. In certain situations, other fine-tuning methods can outperform full fine-tuning, as shown in Fig. 5.

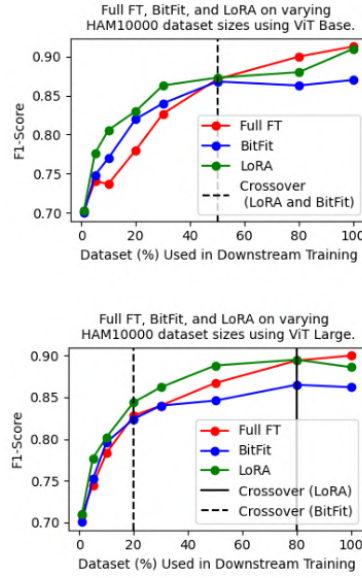


Figure 5: Comparison of fine-tuning methods presented in [13]. BitFit is an adaptive method [34], while LoRA is a selective method [30]

In the experiment that generated Fig. 5, the HAM10000 dataset [35] was used to evaluate the effectiveness of three fine-tuning methods—Full Fine-tuning, BitFit, and LoRA—on two different encoders, ViT Base and ViT Large [36]. F1-Score was measured at various dataset sizes, ranging from the full set of 7,511 images down to just 75 images, with results averaged over three random seeds to ensure reliability.

This experiment demonstrates that parameter-efficient fine-tuning (PEFT) methods like LoRA can improve results compared to full fine-tuning, especially when data is limited. Larger models (ViT Large) benefit from PEFT earlier than smaller ones (ViT Base). While full fine-tuning performs best with abundant data, PEFT methods significantly outperform it as data availability decreases, highlighting the efficiency of these methods when training data and resources are constrained.

During this internship, we designed experiments to evaluate how different fine-tuning methods perform, specifically in the context of diffusion models. These experiments focus on understanding the amount of data required for each method to effectively generalize a distribution, ensuring the generation of reliable images that reflect the underlying data distribution without merely replicating memorized examples.

In the following subsections, we will present the two adaptive methods that we have considered during this project: Low-Rank Adaptation (LoRA) and SVDiff.

2.5.1 LoRA

LoRA is an adaptative method that was introduced by [30].

Neural networks are composed of dense layers (or convolutional layers, or attention layers, etc.) that basically execute matrix operations with weight matrices that have been learned during the training process. According to [37], pre-trained language models have a low "intrinsic dimension", meaning these models can still effectively learn even when subject to a reduction in subspace size.

The authors of LoRA hypothesize that weight updates during downstream task adaptation also have a low "intrinsic rank". The LoRA method involves freezing the pre-trained model's weight matrix of a given layer l $W_0^l \in \mathbb{R}^{d \times k}$ and constraining the weight updates by representing them with a low-rank decomposition:

$$W_0^l + \Delta W^l = W_0^l + AB, \quad (24)$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$. Here, A and B are the trainable parameters, and they are the only parts of the model that receive gradient updates. An illustration of this method is found in Fig. 6. The important point is that r is typically much smaller than d and k (for example, r can be 1, 2, 4, 8, or 64).

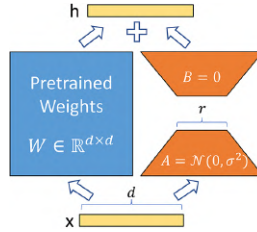


Figure 6: Illustration of the LoRA method presented in [30].

The authors initialize A with random Gaussian values and B with zeros, ensuring that BA is initially zero. Additionally, they scale AB by $\frac{\alpha}{r}$, where α is a constant hyperparameter.

LoRA can be applied to several types of layers (MLP layers, LayerNorm layers, biases, attention layers), although the authors of the original paper only tried it for attention layers (i.e., key, value, and query weights). The choice of r is a training hyperparameter and depends on the dataset. However, based on the authors' experiments and observations, a smaller choice of r (such as $r = 4$) not only results in a smaller number of trainable parameters but often also yields better performance than a larger r (like $r = 64$).

By setting the LoRA rank close to that of the pre-trained weight matrices, the number of trainable parameters increases and LoRA's effectiveness converges to that of fully training the original model. Furthermore, LoRA maintains the same inference latency as a fully fine-tuned model, because the modified weight matrix $W^l = W_0^l + BA$ can be calculated in advance and stored. Task switching is also highly efficient, requiring only minor adjustments that use minimal memory, ensuring there is no additional delay during inference.

LoRA has proven to be highly effective when training data is limited [13], as we will also demonstrate in this project. It is one of the most widely used methods for fine-tuning deep learning models due to its flexibility and efficiency. In fact, LoRA's popularity has led to the development of publicly available implementations, such as those in the PEFT library [38], which allows users to seamlessly integrate LoRA adapters into any layer of any model with just a few lines of code.

2.5.2 SVDiff

SVDiff, introduced by [14], is a method inspired by a prior GAN-based approach (FSGAN) for few-shot adaptation, as proposed by [39]. This method is based on Singular Value Decomposition (SVD) which is a fundamental matrix decomposition technique that expresses any matrix as a product of three matrices [40]. SVD is widely used for various purposes in data analysis, signal processing, and machine learning due to its ability to provide insights into the properties of matrices. Importantly, SVD is guaranteed to exist for any complex matrix, making it a versatile and powerful tool in mathematical applications.

Let's consider a matrix $M \in \mathbb{R}^{n \times m}$. The SVD is a unique matrix decomposition given by

$$M = U\Sigma V^T \quad (25)$$

where $U \in \mathbb{R}^{n \times n}$ and $V \in \mathbb{R}^{m \times m}$ are unitary matrices (i.e., their columns are orthonormal), $\Sigma \in \mathbb{R}^{n \times m}$ is a diagonal matrix with non-negative real entries.

It is not necessary for this report to delve into the details of how to compute the matrices U , V and Σ , since these computations are efficiently implemented in major Python libraries such as PyTorch and NumPy. The crucial aspect to focus on is the interpretation of the matrices U and V , often referred to as the left and right singular vectors, respectively, and the singular values contained in the diagonal matrix Σ . The singular values represent the magnitude of the corresponding singular vectors in U and V , which essentially capture the directions of maximum variance in the data.

The core idea behind SVDiff is to fine-tune only the singular values of the model's weight matrices, while keeping the singular vectors fixed. This selective fine-tuning creates a reduced parameter space referred to as the spectral shift. Similar to LoRA, the goal of restricting the number of trainable parameters is to enhance performance on the target domain.

Consider W as the weight matrix of any layer in the Latent Diffusion Model UNet, which could be a convolutional kernel or the weights of a fully-connected layer. By applying Singular Value Decomposition (SVD) to W , we have:

$$W = U\Sigma V^T$$

where $\Sigma = \text{diag}(\sigma)$ and $\sigma = [\sigma_1, \sigma_2, \dots]$ are the singular values of W arranged in descending order. For fully connected layers, the SVD can be directly applied to the weight matrices. However, for convolutional filters, which have dimensions $W_{\text{tensor}} \in \mathbb{R}^{c_{\text{out}} \times c_{\text{in}} \times w \times h}$, we first need to reshape them to $\text{reshape}(W_{\text{tensor}}) \in \mathbb{R}^{c_{\text{out}} \times (c_{\text{in}} \cdot w \cdot h)}$.

We then update the weight matrix by optimizing the spectral shift $\delta = [\delta_1, \delta_2, \dots]$, which represents the difference between the original singular values and the updated values. The new weight matrix W_δ is defined as:

$$W_\delta = U\Sigma_\delta V^T$$

where $\Sigma_\delta = \text{diag}(\text{ReLU}(\sigma + \delta))$. This process is represented in Fig. 7

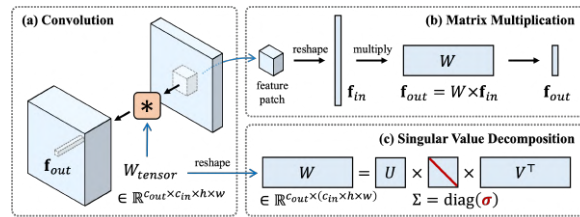


Figure 7: SVD on weight matrices. Image presented in [14].

The approach of fine-tuning only the singular values while keeping the singular vectors fixed effectively reduces the parameter space and maintains model stability, making it particularly useful in scenarios with limited data availability.

In the original paper [14], this method is applied to three tasks different from the one we are considering. Our task consists of shifting the distribution of a model from its pre-trained state to a new downstream task distribution, while SVDiff was primarily presented for:

- **Single Subject Generation:** This task, introduced in the DreamBooth challenge [41], involves fine-tuning a pre-trained text-to-image diffusion model on a single object or concept using only 3-5 images. For

example, if the model generates images of dogs, the aim is to fine-tune it to generate an entire distribution of images based on a specific dog. Fig. 8 provides an example of the challenges associated with this task.

- **Multiple Subject Generation:** Similar to the previous task, but here the goal is to generate multiple distinct subjects within the same scene.
- **Single-Image Editing:** This involves editing a single image based on textual input.



Figure 8: Given a specific clock image (left), it is difficult to generate a reproduction that accurately maintains its key visual features, such as the numbers. This challenge is illustrated in the second and third columns, where DALL-E2 [42] uses image-guided generation and Imagen [7] employs text-guided generation (with the text prompt: “retro style yellow alarm clock with a white clock face and a yellow number three on the right part of the clock face in the jungle”). The method shown in [41], from which this figure was taken, achieves the fidelity displayed in the fourth column. However, as demonstrated in [14], SVDiff can achieve even better results.

For all these tasks, the authors define a weighted prior-preservation loss that is added to the usual loss for diffusion models, presented in Equation 7. This prior-preservation loss ensures that the model retains the distribution learned during pre-training while adapting to the new subject.

As a conclusion, in [14] they found that fine-tuning within the spectral shift parameter space could yield results that are on par with or even superior to those achieved by full weight fine-tuning. Moreover, the spectral shift approach serves as a regularization technique, helping to prevent overfitting, a common issue in our domain. Based on these findings, it is likely that SVDiff could also be effective for our specific task.

3 Implementation

In this section, we will detail the specific architecture and implementation choices for our diffusion model. Additionally, we will explain the development of the image-to-image pipeline used for translation.

3.1 Stable Diffusion Components

As shown in Fig. 4, the main components of a Stable Diffusion Model are the Variational Auto-Encoder, the neural network used for denoising, and any components related to preparing the conditioning class for cross-attention in the denoising neural network.

In the original implementation of Stable Diffusion a text conditioning is processed by a tokenizer and a text encoder. For our task, we removed the tokenizer and text encoder from the model, setting the input corresponding to text in the cross-attention layers to 0. Instead, we use an integer label as the conditioning input, adding this label to the encoded timestep. This new conditioning method is illustrated in Fig. 9.

3.2 Denoiser Architecture

The neural network architecture chosen for $\hat{\epsilon}_\theta$, which predicts the noise in the diffusion model, is a UNet [27]. This choice is particularly suitable because we work with noisy images that have the same dimensions as the original images, and the UNet maintains the same input and output dimensions. A UNet is an encoder-decoder network with skip connections that directly link the encoding layers to their corresponding decoding layers.

The process starts with downsampling, where the input image is gradually reduced in size using convolutional and pooling layers to capture semantic features and reduce computational load. During upsampling, the image is

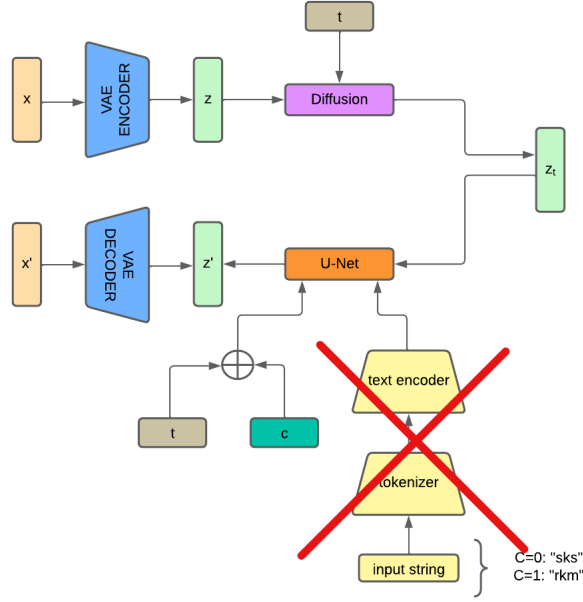


Figure 9: Conditioning of the stable diffusion model used in our implementation. We removed the text encoder and tokenizer, and added a new integer class label that is directly added to the encoded timestep to condition the UNet.

restored to its original resolution with transposed convolutions or upsampling layers. Skip connections help by directly linking high-resolution features from the encoding path to the decoding path, allowing for a precise reconstruction of the image by combining broad, high-level features with fine details. This approach makes UNet ideal for image generation tasks that require preserving both global context and detailed features. The UNet architecture is represented in Fig. 10.

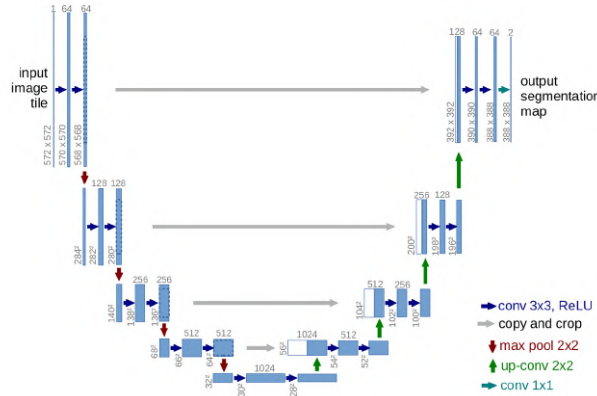


Figure 10: UNet architecture. Image obtained from [27].

For our specific implementation, we used the configuration corresponding to Stable Diffusion 2.1, which is publicly available on Hugging Face. This configuration employs the SiLU activation function, includes four down-sampling blocks with varying output channels and four up-sampling blocks configured similarly but in reverse order. The model also integrates cross-attention layers with multi-headed attention and uses group normalization. Importantly, the UNet is also conditioned on the encoded timestep, incorporating temporal information to dynamically adjust its predictions as the diffusion process progresses.

Instead of using the pre-trained weights from Stable Diffusion 2.1, we used the weights from Stable Diffusion Nano, available on Hugging Face. While Stable Diffusion 2.1 was trained to generate images at a resolution of 768x768, Stable Diffusion Nano was initialized with the weights from this version but further trained for an

additional 300,000 steps at a resolution of 128x128, so Stable Diffusion Nano specifically adapts the model to this size. This lower resolution was chosen to better match our datasets, allowing for improved adaptation and performance in our specific image generation tasks. The UNet is easily initialized using PyTorch with the given configuration and weights file.

3.3 Training script

For implementing the training script, we reused a script from a GitHub repository containing the code for SVDiff. This script was originally designed to fine-tune a pre-trained model using SVDiff for the task of single subject generation conditioned on text, as described in section 2.4. To adapt it to our specific task, we introduced several modifications, such as:

- Since preserving the previous distribution is not needed for our task, we do not use the prior-preservation loss described in Section 2.5.2.
- We added a parameter to select the fine-tuning method to train the model for the downstream task. We defined five main methods:
 - **SVDiff**: Already implemented in the original code, but we had to reimplement it for compatibility issues with Stable Diffusion 2.1 (the original code was for Stable Diffusion 1.5).

As described in Section 2.5.2, this method involves performing SVD on specific weight matrices (in our implementation, we apply it to every learnable weight matrix, including 1D convolutions, 2D convolutions, and linear layers), creating a new parameter δ added to the singular values of this decomposition, and freezing all other parameters except δ .

This new δ parameter was directly integrated into the definitions of these layers in the UNet implementation for stable diffusion models, available in the repository Hugging Face Diffusers, with release 0.14.0.

 - **Full Fine-Tuning**: This method initializes the weight matrices with those of the pre-trained model and then updates every weight during training.
 - **From Scratch**: Similar to the previous method, but this time initializing the weights randomly, without using the pre-trained weights.
 - **LoRA**: Uses LoRA adapters for each weight matrix in the UNet, as described in section 2.5.1.
 - **Attention Fine-Tuning**: This method updates only the weights corresponding to the projection matrices W_Q^i , W_K^i , and W_V^i , as presented in Equations 20, 21, and 22, while freezing all other parameters.

It is important to mention that the training script leverages the Hugging Face Accelerate library to facilitate training, as it enables fast and convenient parallelization of model training across multiple GPUs, a crucial aspect of our setup since we needed to use many GPUs simultaneously.

3.4 Image-to-image Pipeline for Image Translation

As discussed in Section 2.3, DDIMs have the unique property of being invertible, allowing us to generate a specific Gaussian seed x_T that corresponds to a given image x_0 . Leveraging this property and emulating the method presented in [43] (that was presented for two unconditional diffusion models, but we can also do it with a single two-class conditional diffusion model), we can learn an unpaired mapping between two class distributions. This technique is straightforward and illustrated in Fig. 11. Suppose we have an image x_0 belonging to class $c = 0$, and we want to translate this image to class $c = 1$, retaining as many high-level features as possible while making the necessary changes to switch classes. The algorithm is presented in Alg. 3.

Although the Python library ‘diffusers’ provides a pipeline for DDIM generation that can be used to easily perform the backward process for our translation task, and includes an implemented inverse DDIM scheduler, there is no existing pipeline for executing the inverse process directly. Therefore, we had to implement this ourselves by modifying the code of the DDIM pipeline, which is publicly available in this GitHub repository.

Algorithm 3 Class Translation using DDIM Inversion

- 1: **Input:** Image x_0
 - 2: Apply the inverse diffusion process to obtain the noise $\hat{x}_T$ that would generate x_0 during the reverse process.
 - 3: Initialize $c = 1$, the target class for translation.
 - 4: Apply the backward diffusion process on $\hat{x}_T$, conditioned on c .
-

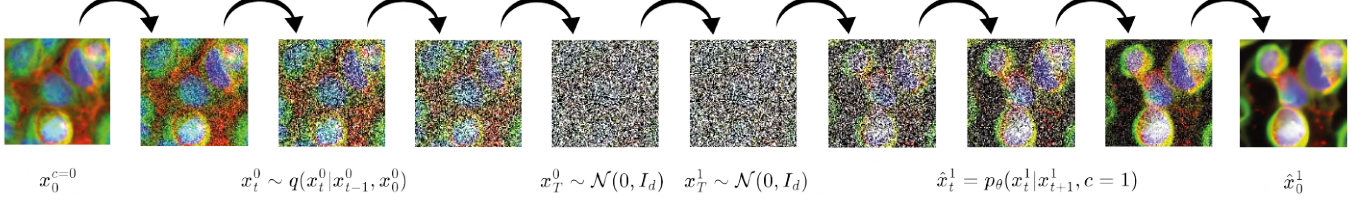


Figure 11: Schema of the DDIB method for image-to-image translation between the DMSO and latrunculin classes from the BBBC021 dataset.

4 Experiments

In this section, we describe the experimental setup and methodologies used to evaluate our models. We first introduce the datasets employed in our experiments, followed by the validation and testing metrics that were chosen to assess model performance. We then provide a detailed overview of the experiments conducted to test various configurations and fine-tuning strategies.

4.1 Datasets

In this project, we used two different datasets:

- **BBBC021:** This dataset, provided by the Broad Institute, consists of fluorescent microscopy images of breast cancer cells treated with 113 different molecules, each treatment at eight different concentrations. In our case, we used only the images corresponding to the treatment with Latrunculin B at the two highest concentrations, along with control samples treated with DMSO. These images are RGB and contain several cell nuclei, although we cropped a 128×128 region around each cell nucleus to obtain single-cell images.

Therefore, we consider two classes: *DMSO* and *latrunculin_B_high_conc*. For each class, we have 1685 images at a resolution of $128 \times 128 \times 3$. Examples of these images for each class can be seen in Fig. 12.

- **Organoid images:** This dataset was provided directly by Valérie Mezger’s lab. It contains microscopy images of organoids, which are complex 3D structures. The three-dimensional nature of these cellular formations often results in portions of the images appearing unclear or out of focus. The dataset contains two classes corresponding to two hypothesized different phenotypes: *IMR*, representing healthy organoids, and *RSTS*, representing organoids with an RSTS mutation. The differences between these two classes are very subtle and it is hard to spot them even for experts.

Although the original dataset images have a resolution of $2200 \times 2699 \times 3$, we downsampled to a resolution of $512 \times 512 \times 3$ from each image to alleviate computational costs. An intelligent method for filtering out-of-focus images or better downsampling technique is recommended for future work.

With these modifications, our dataset contains 56 images of the *IMR* class and 85 images of the *RSTS* class, each with a resolution of $512 \times 512 \times 3$. Examples of these images can be seen in Fig. 13.

4.2 Validation metrics: FID

One of the most commonly used metrics to determine the quality of images created by a generative model is the Fréchet Inception Distance (FID), which was used throughout the experiments, even though it has some limitations [44, 45].

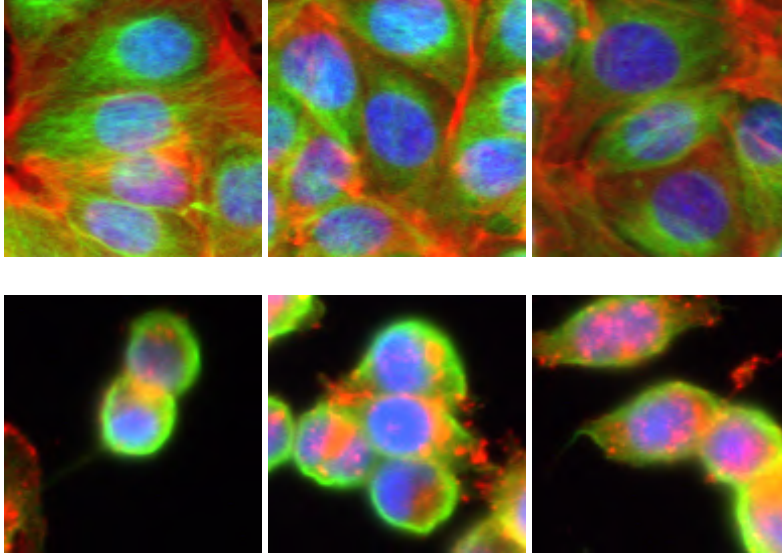


Figure 12: Examples of the cropped $128 \times 128 \times 3$ images from the BBBC021 dataset used for this project. The first row contains images from the *DMSO* class, while the second row contains images from the *latrunculin_B_high_conc* class.

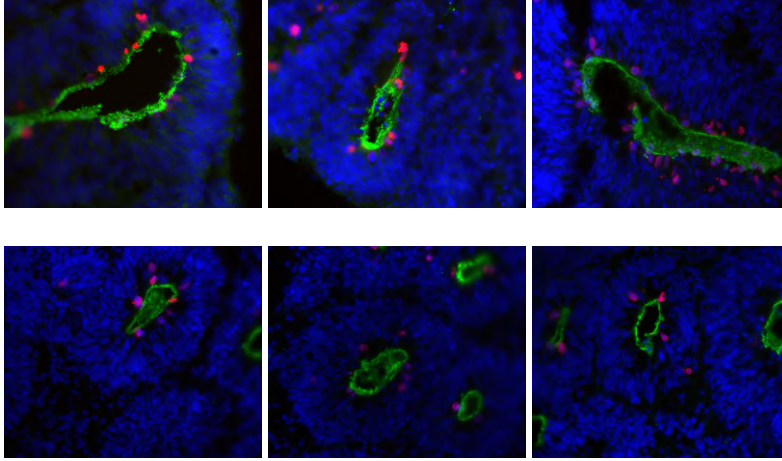


Figure 13: Examples of the cropped $512 \times 512 \times 3$ images from the organoids dataset. The first row contains images from the *IMR* class, while the second row contains images from the *RSTS* class.

The Fréchet Inception Distance (FID) is a metric used to evaluate the quality of images generated by generative models. It was first introduced in [45] for assessing GANs and it measures the similarity between the distributions of generated images and real images in the feature space of a pre-trained Inception v3 network [46]. The lower the FID score, the closer the generated images are to the real images in terms of visual quality. To compute the FID, the following algorithm is used:

Algorithm 4 Compute Fréchet Inception Distance (FID)

Require: Generated images $\{x_{\text{gen}}^i\}$, Real images $\{x_{\text{real}}^i\}$

Ensure: FID score

- 1: Load pre-trained Inception v3 model $\mathcal{I}$
- 2: Extract features $\mathbf{F}_{\text{gen}} = \{\mathcal{I}(x_{\text{gen}}^i)\}$ and $\mathbf{F}_{\text{real}} = \{\mathcal{I}(x_{\text{real}}^i)\}$ for all images
- 3: Compute means $\mu_{\text{gen}}, \mu_{\text{real}}$ and covariances $\Sigma_{\text{gen}}, \Sigma_{\text{real}}$ from $\mathbf{F}_{\text{gen}}$ and $\mathbf{F}_{\text{real}}$
- 4: Calculate FID score:

$$\text{FID} = \|\mu_{\text{gen}} - \mu_{\text{real}}\|^2 + \text{Tr}(\Sigma_{\text{gen}} + \Sigma_{\text{real}} - 2(\Sigma_{\text{gen}}\Sigma_{\text{real}})^{1/2})$$

- 5: **return** FID score
-

The FID has remained the standard metric in the field for several years, and there is empirical evidence of a strong correlation between FID scores and the perceived visual quality of generated images [47].

However, this metric has some clear drawbacks:

- The metric is heavily dependent on the Inception network, an older architecture, to accurately capture the features of images. Also, the Inception network was trained on natural images, which can belong to a distribution significantly different from biological images.
- A model that merely replicates the training data could achieve a perfect (zero) score.
- It is crucial to note that the FID is biased towards higher values when computed with fewer samples, and this bias becomes more pronounced with a smaller number of generated images (particularly in the range of a few thousand).

Even though our datasets consist of only a few thousand images, we still use the FID as our metric. During each different training run, we generate as many images as there are in our entire original dataset to compute the FID of the model at regular intervals of epochs. This FID calculation compares, for each class, the distribution generated by the model to our entire original dataset, allowing us to assess how closely the model approximates the original distribution. As detailed in Section 4.4, our training set is a subset of the original dataset. However, for validation purposes, we compare the generated images with the entire dataset. Since FID calculations require several thousand images to provide meaningful comparisons, using a small validation set would not be effective, especially considering we are already constrained by the number of available images.

By computing FID in this manner, we aim to evaluate how closely each training method can approximate the entire original dataset. Additionally, we seek to verify that the FID converges to a specific value during training, indicating that the model is learning some distribution—whether or not it is the desired one, which will be further examined using the testing method proposed in Section 4.3.

4.3 Testing Generalization Capabilities

To assess a trained model’s ability to generate images from a given distribution, analyzing the quality of the generated images alone is insufficient. High-quality images might simply indicate that the model has memorized images from the training set and is reproducing them without the ability to generate diverse outputs.

When considering the generalization capabilities of a diffusion model—specifically its ability to generate images that are not only similar to those in the desired distribution but also diverse—there are three distinct regimes of generalization, depending on the size of the training dataset [48, 49].

- **Memorization Regime:** If the training dataset is too small, the model may fall into a regime of memorization, where it tends to replicate images it has already seen during training. This leads to overfitting, as the model is unable to generalize beyond the limited examples it was exposed to.
- **Transition Regime:** As the size of the training dataset increases, the model may enter a transition regime. In this phase, the model starts to introduce more diversity in the generated images, moving away from merely reproducing memorized images. However, the quality of these images may still be low, as the model has not yet been exposed to enough data to fully learn the underlying distribution.
- **Generalization Regime:** When the training dataset is sufficiently large, the model can reach the generalization regime. Here, the model has learned the full distribution represented by the training data, allowing it to generate images that are both high-quality and diverse, effectively capturing the variability within the desired distribution.

Thus, for each method, if we train different models using varying training dataset sizes, we might be able to determine the regime in which each model operates by examining the generated images.

For smaller training datasets, we may observe that the model generates images with decent quality. However, when we search for the closest images in the training dataset using cosine similarity, it becomes apparent that the generated image is essentially a reproduction of a closely matched training image. As we increase the size of the training dataset, the generated images are less similar to any single training image, but the overall quality of

the generated images may decrease. Finally, with a sufficiently large training dataset, the model may produce high-quality images that are not overly similar to any single training example. As we will see in Section 5.4, our results align with these expectations, which can help us estimate the number of images required to reach the generalization regime for each method.

In addition to this qualitative analysis of the generated images and their differences from the closest training images, we use a numerical indicator to better understand the generalization capabilities of the model. For this purpose, we employ the same type of histograms that were used in [49]. The idea is to, for a given fine-tuning method and training dataset size, train two separate models using two training datasets of the same size with no overlapping data.

For each of the two trained models and for each class, we generate M images using the same random seed. We calculate the correlation between each pair of images generated by the two models from the same Gaussian noise and plot these correlations in a histogram. We expect to see low correlations during the memorization regime, more uniform correlations in the transition regime, and correlations close to one in the generalization regime.

Additionally, for one of these models, we find the closest image in the original dataset for each generated image and compute the correlation with this closest image. By plotting these correlations in another histogram, we expect to see the majority of correlations being one during the memorization regime, fewer correlations being one during the transition regime, and correlations significantly lower than one during the generalization regime. However, these correlations will not be zero either; they will likely still be high since even the closest image, while different, may still have a strong correlation due to the consistent nature of cell images.

4.4 Conducted Experiments

Now, we will detail the experiments conducted to compare the different fine-tuning methods. All experiments were performed on the JeanZay supercomputer, using 4 V100 GPUs with 32 GB of memory each. The resolution of all images was consistently set to 128x128x3 (except for the organoids experiment). For training our models, we employed a cosine learning rate scheduler with no warmup steps, starting with an initial learning rate of 1e-3. For image generation, we used 100 generation steps with the accelerated sampling method described in Section 2.2. Each model was trained for a maximum of 30,000 steps to ensure sufficient time for convergence, irrespective of how quickly each method reaches convergence. Note that the number of trainable parameters varies significantly between the fine-tuning methods used in our experiments, as shown in Table 1.

Fine-Tuning Method	Number of Trainable Parameters
SVDiff	220,000
LoRA	420,000
Attention Fine-Tuning	100,000,000
Full Fine-Tuning	780,000,000
From Scratch	780,000,000

Table 1: Number of trainable parameters for each fine-tuning method.

As we will see in Section 5.1, we first trained a model from scratch using the entire dataset to verify that this method is unable to learn the distribution with the available data.

Next, to examine the convergence of the FID (Fréchet Inception Distance) and compare FID values across the four different methods (LoRA, SVDiff, attention fine-tuning, and full fine-tuning), we defined four different dataset sizes (per class): 10, 50, 100, and 1000. This setup allows us to observe whether the model converges to a meaningful solution even when using very few images, and to compare how closely the generated images resemble the original ones when using each method with different training sizes. For each method and each training size, we repeated the experiment three times with different datasets of the specified size to ensure that the results are not due to random chance in the selection of images.

We then assessed the generalization capabilities of the different methods as described in Section 4.3. For each model, we used pairs of trained models with training sizes of 10, 50, 100, and 800. Note that we could not reuse

the models trained with 1000 images for the FID analysis, as we required that the models have no overlap in their training sets, which was impossible with two datasets of 1000 images selected from a total of 1685 images.

In addition, we applied our image-to-image translation pipeline to qualitatively assess the quality of the translations. No numerical analysis was performed because, although it would be highly relevant, there was not enough time during the internship to complete this task.

Finally, we trained a generator on the organoids dataset. However, the initial results have not been satisfactory, and further experiments will be necessary to achieve better outcomes with this dataset.

5 Results

In this section we will present the results for each of the experiments presented in Section 4.4.

5.1 From Scratch Method

We first conducted a full training from scratch using all available data from the BBBC021 dataset. The images generated by the resulting model are shown in Fig. 14. It is evident that the quality of these generated images is very bad. This experiment was conducted early in the internship, and the poor quality of the images justifies discontinuing this method in favor of the other four methods described in Section 3.2.

Surprisingly, the model could not achieve good generation results. The model is quite large, making it difficult to converge, especially with the limited amount of data available. It is important to note that the space of trainable parameters is vast, and our dataset is not sufficient to cover this space adequately. It might also be a matter of time, as more training time could have been necessary to achieve better results.

Although we used the same or even smaller amounts of data for other methods, those methods benefit from initializing the model with pre-trained weights from Stable Diffusion. This pre-training likely provides a significant advantage by allowing the model to leverage previously learned features, such as shapes, colors, and textures, to more effectively adapt to the new distribution.

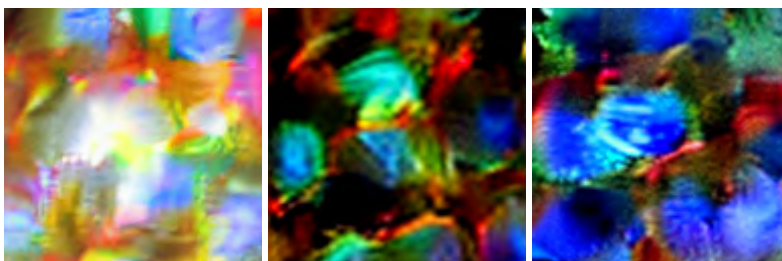


Figure 14: Examples of images from the *DMSO* class, generated using a model trained from scratch with the entire BBBC021 dataset.

5.2 FID Convergence

Let’s analyze the FID scores obtained during the training of the models using different fine-tuning methods and various training subsets, as described in Section 4.2. In Fig. 15, we present the results for one training session per training dataset size for each method.

For both SVDiff and LoRA, we observe a smooth convergence of the FID scores across iterations. The models generally converge within 5000 iterations for all training sizes, except when using 1000 training samples in SVDiff, where convergence takes around 10,000 iterations. Additionally, the FID scores are higher when using only 10 training samples compared to 50, and the lowest FID scores are consistently achieved when using 100 or 1000 training samples. We do not observe any significant differences between the performance of LoRA and SVDiff. These results suggest that the model is consistently able to converge to a distribution that becomes closer to the target distribution as the number of training samples increases, although this closeness might still result from memorization. The similarity in FID scores between using 100 and 1000 training samples does not

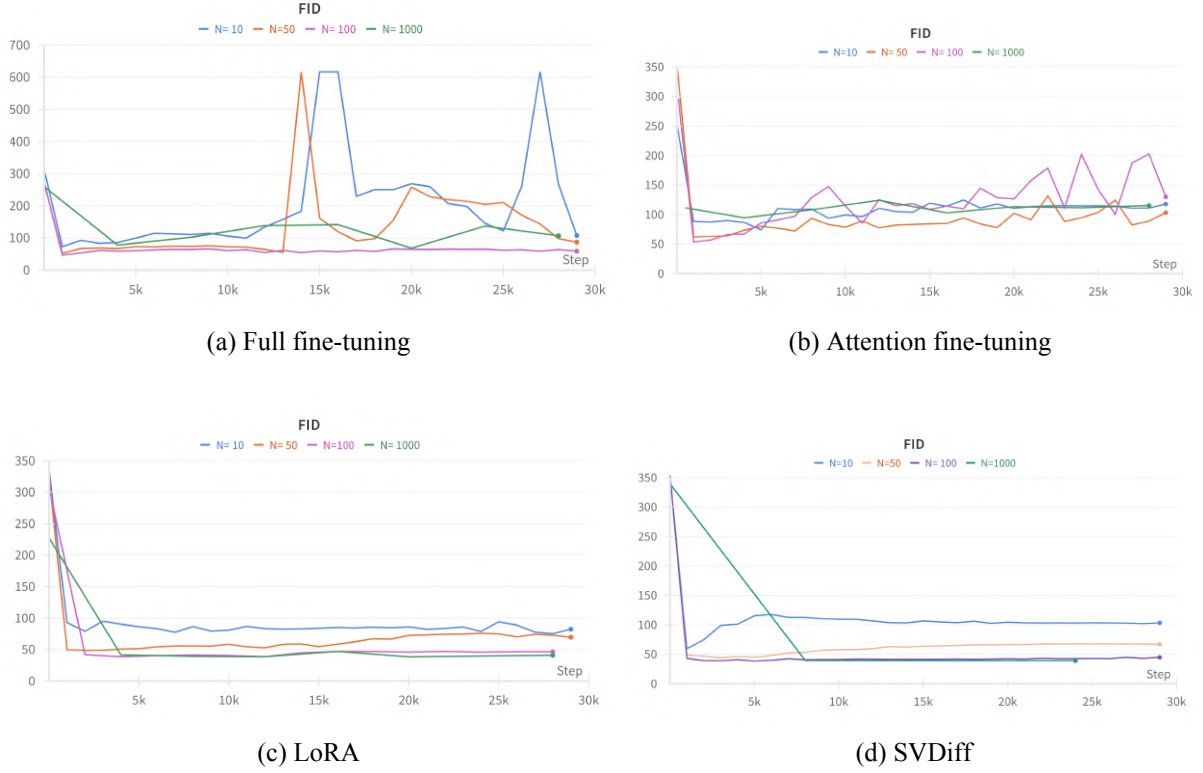


Figure 15: Graphics showing the evolution of the FID score for each method. For each method, we show results from four different training sizes per class: 10, 50, 100, and 1000.

necessarily indicate that training with 100 images is sufficient to reach the model’s generalization regime. It is possible that both distributions are equally close to the original distribution, but the individual image quality and generalization capability may differ.

For attention fine-tuning, the convergence is less smooth, but there is still evidence of convergence to a distribution in all training sessions. In this case, it is not easy to find a direct correlation between the number of training samples and the FID score achieved. Lastly, with full fine-tuning, the model converges to a distribution when using 100 and 1000 training samples, but it fails to converge to a specific distribution when using only 10 or 50 training samples. It is surprising that full fine-tuning achieves lower FID scores when using 100 images compared to 1000 images. However, as we will demonstrate in Section 5.4, this occurs because with 100 samples, the method is still memorizing the training images. In contrast, with 1000 images, the model enters the transition phase, where the generated images are of very poor quality, leading to a higher FID score.

Both attention and full fine-tuning methods are able to achieve lower FID scores than LoRA and SVDiff, but the fact that training is more abrupt with these methods may suggest that the models are attempting to memorize the data rather than learning the underlying distribution.

We can complement the analysis of the FID scores with Fig. 16, which shows the evolution of the training loss for each method and training size considered.

These results reaffirm what we observed in the evolution of the FID scores in terms of convergence. Every training appears to converge to some distribution, except when using full fine-tuning with 10 or 50 samples. When interpreting these loss graphs, we should keep in mind that many models are likely to suffer from overfitting due to the small size of the datasets. Therefore, a low loss does not necessarily indicate a high-quality model.

Finally, it is important to note that we have shown only a single training instance for each method and training size here. However, as mentioned in Section 4.2, we actually performed three separate training runs for each configuration. In almost every configuration, the results from all three training runs are very similar, as demonstrated in Fig. 17. This consistency indicates that the observed convergence depends primarily on the number of training samples rather than the specific choice of samples.

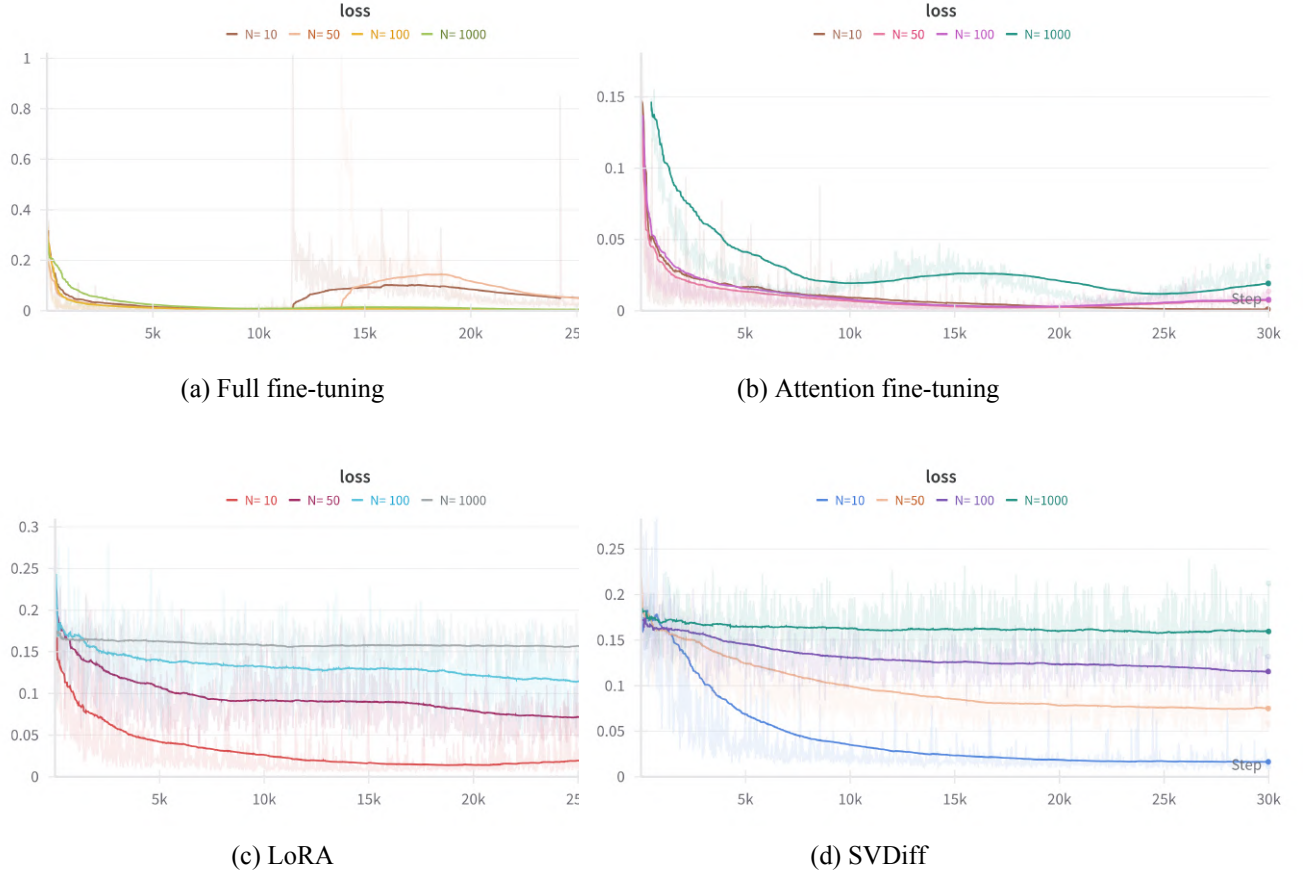


Figure 16: Graphics showing the evolution of the training loss for each method. For each method, we show results from four different training sizes per class: 10, 50, 100, and 1000.

Nevertheless, this is not the case when using the full fine-tuning method with 10 or 50 training samples, as shown in Fig. 18. This aligns with the fact that these were the only cases where the model did not converge, suggesting that training with the full fine-tuning method is more sensitive to the specific selection of training samples when the dataset is small.

5.3 Image Quality Analysis

In this subsection, we take a moment to analyze the quality of the generated images. For example, let’s examine the images generated for each method and training size in Table 3, where all images were produced using the same initial Gaussian noise. Additionally, for each training size, every model was trained using the same dataset.

At first glance, the quality of the images generated by models trained with just 10 images appears to be quite high across all methods, although slightly lower when using full fine-tuning. The generated images for this training size are very similar in each case, which could suggest two possibilities: either all models are in the memorization regime and are simply reproducing a memorized image from the training set (which is the same for all models), or they have all reached the generalization regime with just 10 images.

However, it is more likely the first scenario, since when using models trained with 50 samples, all methods generate very different images. It would not be logical to assume the models are in the generalization regime with 10 images but not with 50. The quality of the images generated by the models trained with 50 samples is generally poor, except for those produced by the full fine-tuning method. The image generated by the attention fine-tuning method is particularly poor.

When using models trained with 100 samples, the generated images remain quite different from each other. For this training size, full fine-tuning continues to produce high-quality images, while the attention method still

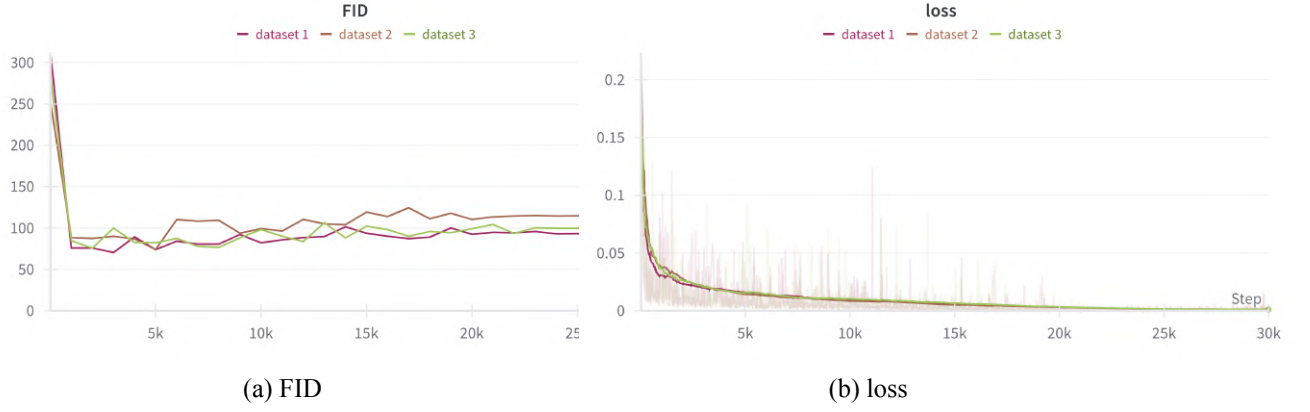


Figure 17: Graphics showing the evolution of the FID score and training loss when fine-tuning the dataset using the attention method with a training size of 10 images per class. Each graphic shows results from three separate trainings using three non-overlapping datasets of this size.

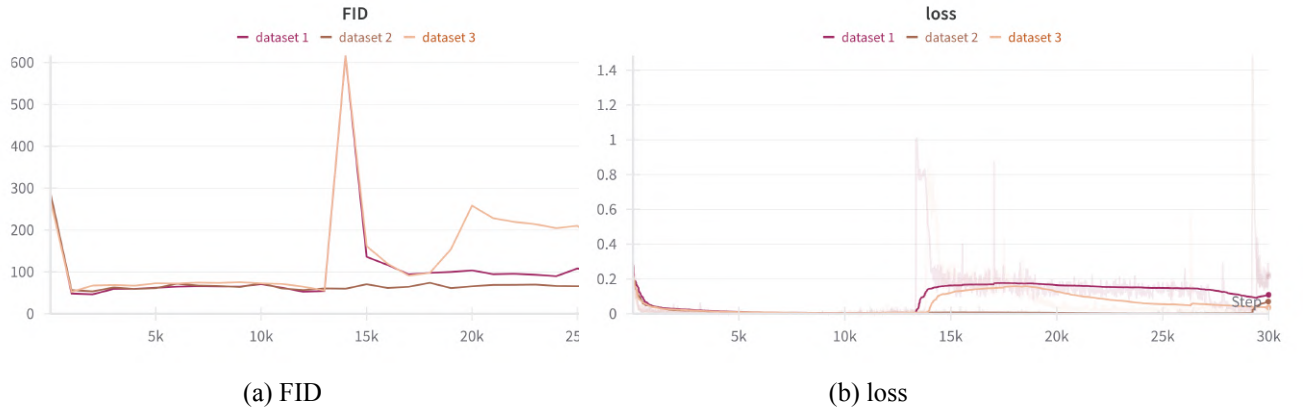


Figure 18: Graphics showing the evolution of the FID score and training loss when fine-tuning the dataset using the full fine-tuning method with a training size of 50 images per class. Each graphic shows results from three separate trainings using three non-overlapping datasets of this size.

generates the lowest quality images, although there is some improvement compared to models trained with 50 images. SVDiff and LoRA also show improved quality compared to their 50-sample counterparts, but the generated images still do not match the quality of the original images.

Finally, with models trained using 1000 samples, LoRA and SVDiff generate nearly identical images with very high quality. The attention method produces an image somewhat similar to those generated by LoRA and SVDiff, but with significantly lower quality. Full fine-tuning, on the other hand, generates an image that is quite different from the others and of very poor quality.

One hypothesis to explain these results could be that all models are in the memorization regime when trained with only 10 samples. When training with 50 samples, full fine-tuning remains in the memorization regime, while the other methods enter the transition regime, which explains the lower quality of the images, with attention fine-tuning showing the worst quality because it is furthest from the generalization regime compared to SVDiff and LoRA. With 100 samples, this trend continues, although image quality improves as each method gets closer to the generalization regime. With 1000 samples, full fine-tuning enters the transition regime, LoRA and SVDiff reach the generalization regime (explaining why they produce similar high-quality images), and attention fine-tuning is almost in the generalization regime but requires more data to fully reach it. This hypothesis will be further examined in Section 5.4.

More examples of generated images can be found in the Appendix, in Table 4 and Table 2.

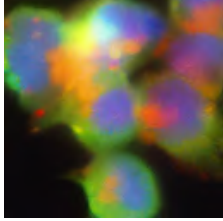
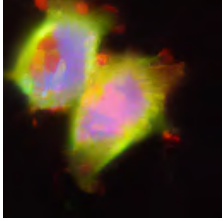
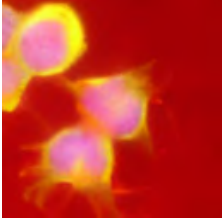
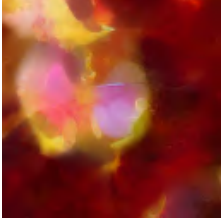
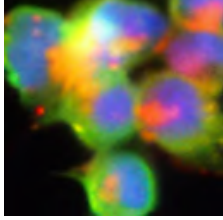
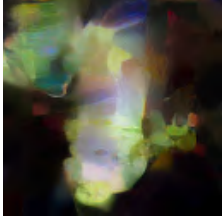
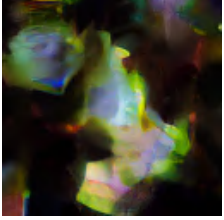
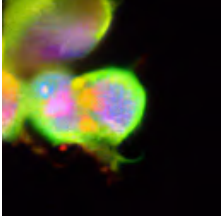
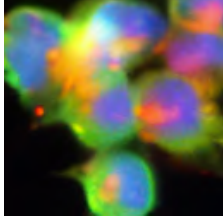
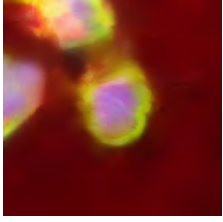
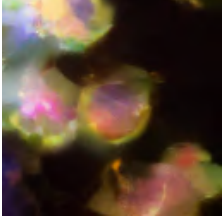
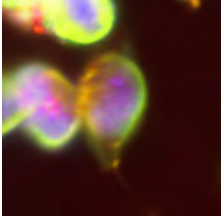
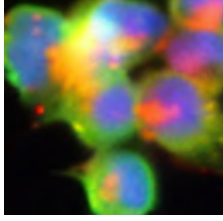
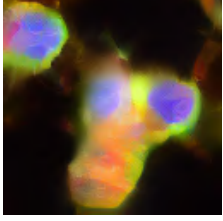
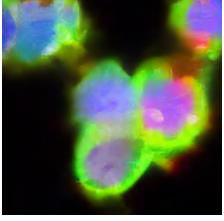
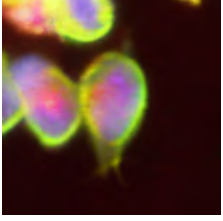
Method	N = 10	N = 50	N = 100	N = 1000
Full fine-tuning				
Attention fine-tuning				
LoRA				
SVDiff				

Table 2: Examples of images of the class *latrunculin_B_high_conc* generated using four different methods, each trained with four different training sizes per class. All images were generated with the same Gaussian noise.

5.4 Generalization Capabilities

To more accurately assess the generalization capabilities of each trained model, we generated the histograms described in Section 4.3 for each class. The histograms for the class *latrunculin_B_high_conc* are presented in Fig. 19.

By examining these histograms, we can confirm the hypothesis proposed in Section 5.3. Specifically, for the full fine-tuning method, training with 10, 50, 100 or 1000 samples places the model in a memorization regime, as evidenced by the high correlation between generated images and their closest counterparts in the training set. Additionally, images generated by two different models trained on non-overlapping datasets exhibit very low correlation.

For attention fine-tuning, we clearly observe a memorization regime when training with 10 or 50 samples. With 100 samples, the model begins to enter the transition regime, as indicated by an increase in correlation between images generated by different models and a decrease in correlation with the closest training images, although the model is still primarily in the memorization regime. When trained with 1000 images, the model starts to move further into the transition regime. Despite still having many memorized images, the higher correlation between images generated by different models and the presence of non-memorized images suggest an ongoing shift towards generalization.

SVDiff and LoRA exhibit similar behavior. Both methods are clearly in the memorization regime when trained with 10 samples, enter into the transition regime with 50 and 100 images, and reach the generalization regime when trained with 800 images.

After computing the histograms, we also saved the closest training image for each generated image. In Fig. 20,

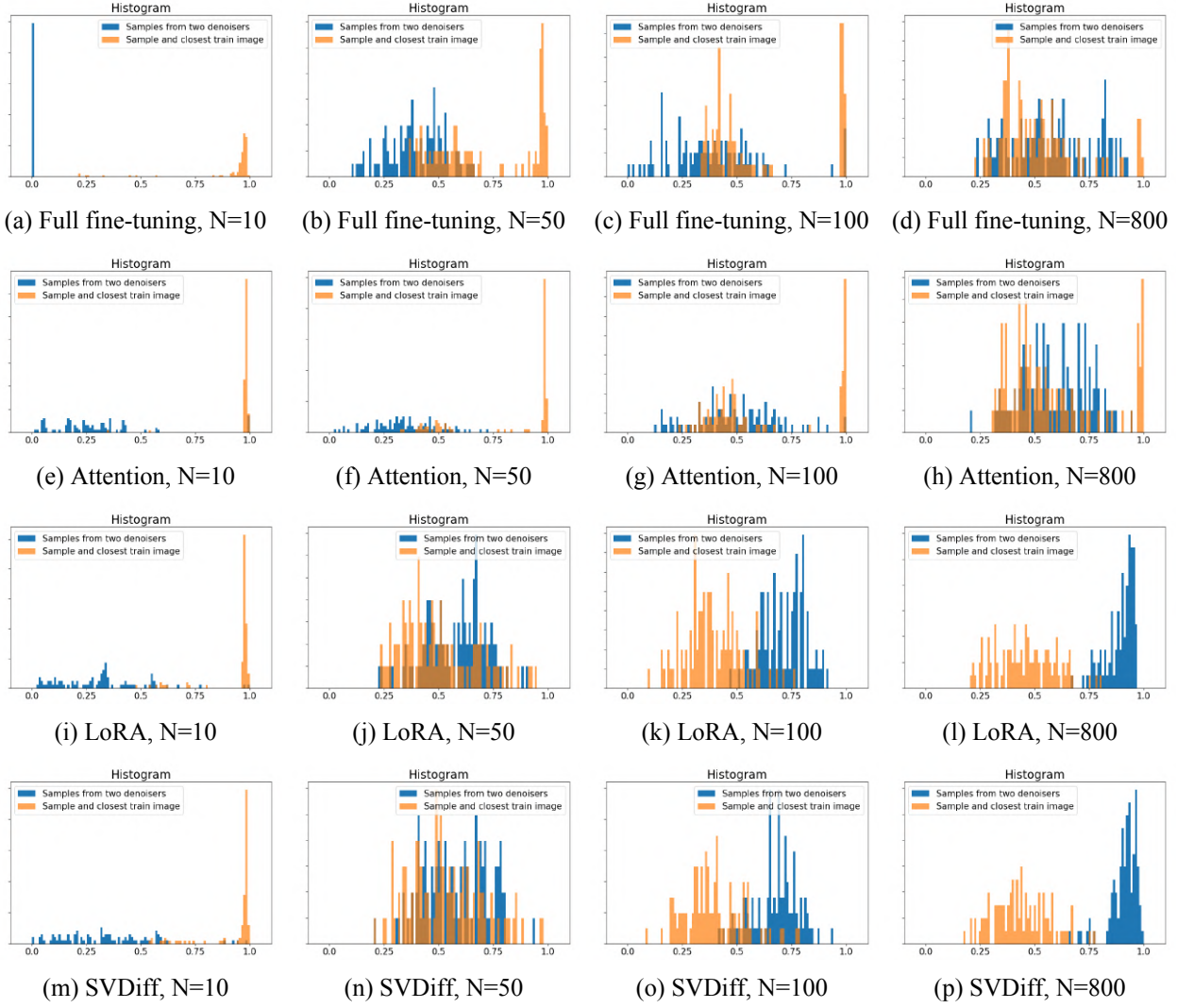


Figure 19: Generalization histograms for class *latrunculin_B_high_conc* as described in Section 4.3. The orange bars represent the correlations between images generated by the same seed from two models trained on different non-overlapping datasets. The blue bars represent the correlations between the generated images and their closest images in the training dataset.

we display the most similar pair of generated and closest training images among all obtained pairs, based on cosine similarity distance.

These findings reinforce our observations from the histograms. Full fine-tuning consistently memorizes at least one image until trained with 800 images, at which point it enters the transition regime. Attention fine-tuning also always memorizes at least one image. In contrast, LoRA and SVDiff memorize at least one image when trained with 10 or 50 samples. While we have noted that they are in the transition regime when training with 50 images, it is still typical for some images to be memorized, although the majority are not.

Similar results are observed for the class *DMSO*, as shown in the appendix in Fig. 25 and Fig. 24. These conclusions suggest that 800 images are insufficient to reach the generalization regime when using attention or full fine-tuning. However, SVDiff and LoRA reach a model capable of generalizing the distribution somewhere between 100 and 800 images.

It is important to note that we are always considering small datasets in this analysis. With sufficient data, full fine-tuning and attention fine-tuning would presumably achieve better results due to their larger number of trainable parameters. However, within the scope of this project, we focus on small datasets, and therefore, we do not explore the point at which attention and full fine-tuning become preferable to LoRA and SVDiff.

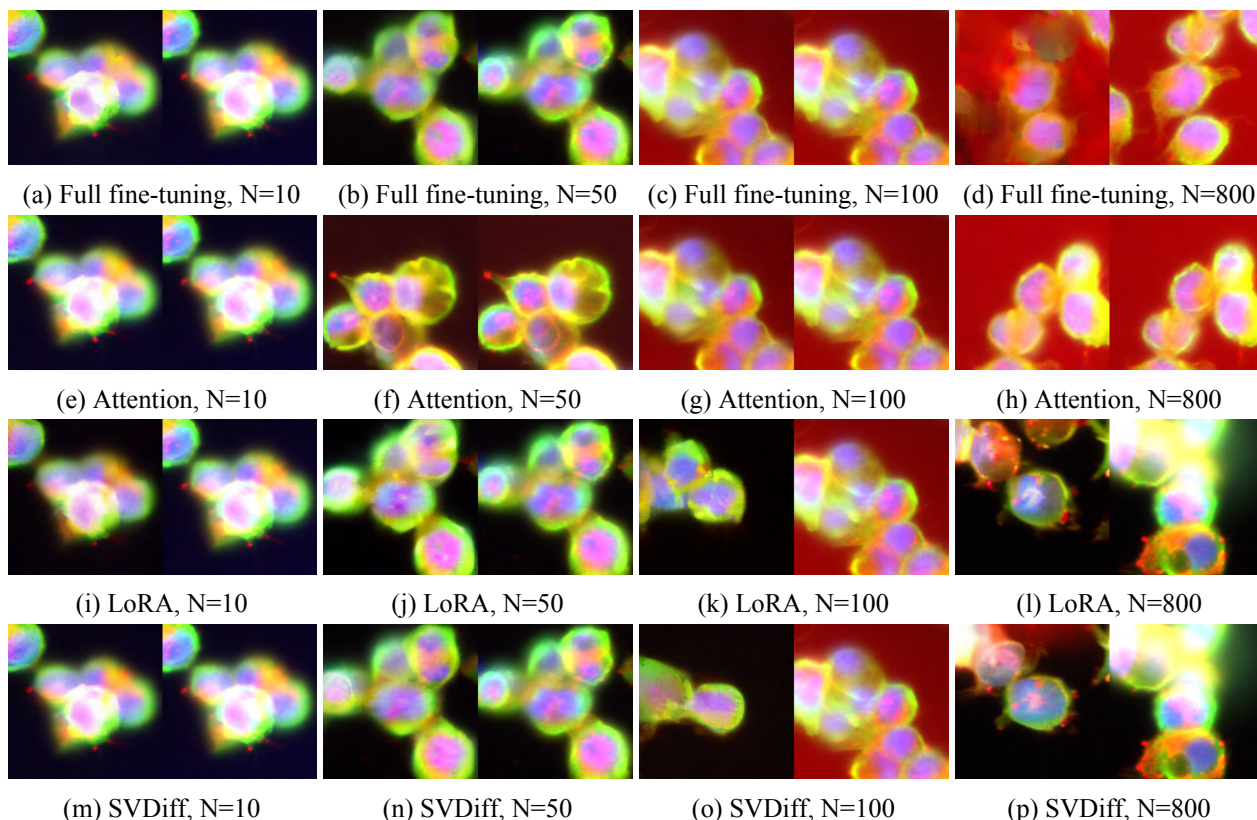


Figure 20: Top-1 *latrunculin_B_high_conc* most similar generated and training image for each model, according to cosine similarity distance. For each pair, the generated image is on the left and the closest training image is on the right.

5.5 Images Evolution During Training

In this subsection, we will explore how many optimization steps are required to achieve a distribution shift using the model with the best results (specifically, the model trained with the SVDiff method using 1000 training samples. We could have also used the corresponding model fine-tuned with LoRA).

To this end, we saved the generated images using the same Gaussian noise every 70 optimization steps, and observed that the model appears to achieve the distribution shift after approximately 350 steps. The images generated throughout this process are displayed in Fig. 21.

5.6 Image-to-Image Translation

In this experiment we demonstrate that the image-to-image translation pipeline produces high-quality translations using the same model referenced in the previous section. Due to time constraints, we conducted only a qualitative analysis. Some examples of these translations are shown in Fig. 22. As illustrated, the translation task is correctly performed, as the translated images retain as many high-level features as possible when changing classes, such as shape, size, and orientation.

Notably, there is a decrease in the number of cells when translating from *DMSO* to *latrunculin_B_high_conc*, which is expected due to the toxicity of the *latrunculin_B_high_conc* treatment. This compound is known to kill cells, resulting in fewer cells being visible in images from this class.

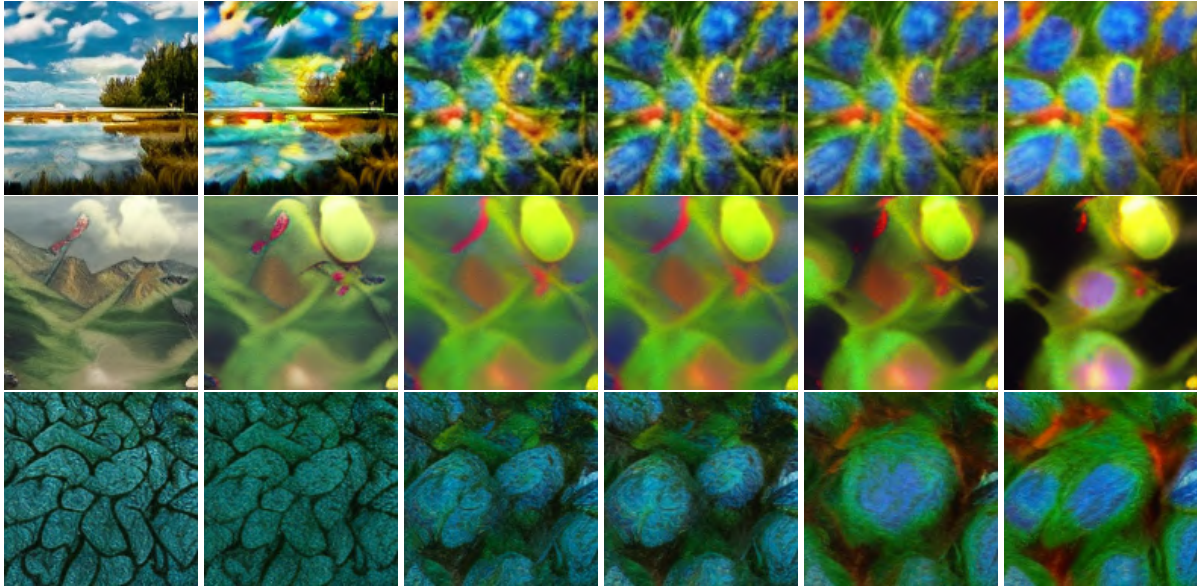


Figure 21: Evolution of images generated by the model during different iterations of training, using a model fine-tuned with LoRA and 1000 images. The images in each row are generated using the same seed, with the image on the left being generated after 1 iteration and the image on the right after 500 iterations.

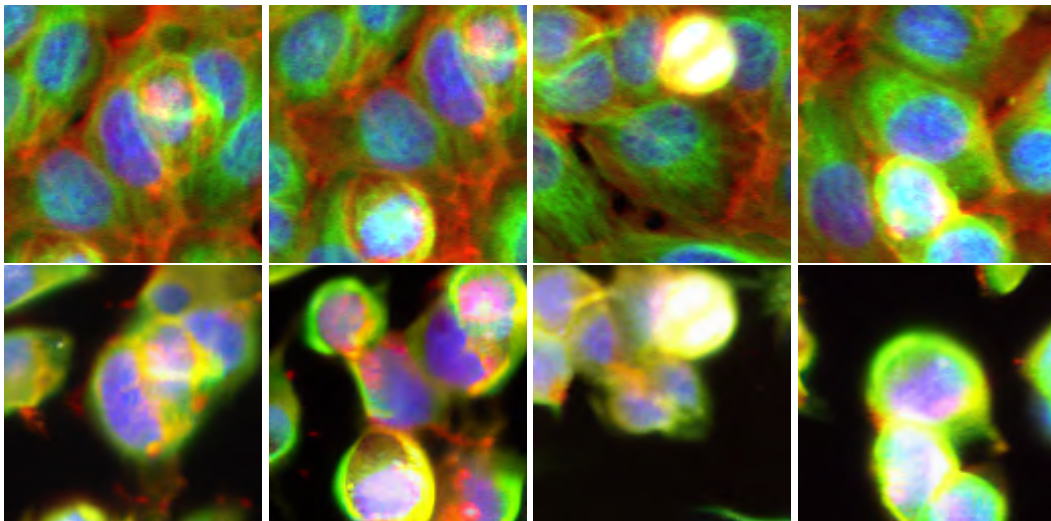


Figure 22: Examples of results achieved with our pipeline for image-to-image translation using a model fine-tuned with the SVDiff method and 1000 images per class. The first row shows the original images from the *DMSO* class, while the second row displays their respective translations to the *latrunculin_B_high_conc* class.

5.7 Organoids Images Generation

Our final experiment involved training a model using the organoids dataset. As shown in Fig. 23, where we present images generated by a model trained on all 141 samples from the organoids dataset and fine-tuned using LoRA during 30000 optimization steps, the results are qualitatively unsatisfactory. Several factors may have contributed to this:

- **Image downsampling:** We reduced the resolution of the images, leading to a significant loss of detail, which may have removed crucial information necessary for distinguishing between classes. A more intelligent approach to downsampling or the use of the original image resolution (which is currently intractable for our lab) is recommended.
- **High variability:** The organoids dataset contains substantial variability in the images, which adds complexity to the task.

- **High resolution:** Even after reducing the resolution, the organoids dataset still retains a higher resolution compared to the BBBC021 dataset. This inherently requires more data and longer training times, which contributes to the difficulty in achieving good results.
- **Limited Data** We observed suboptimal results with only 50 images in the BBBC021 dataset, and we have a similarly limited dataset here, with barely 50 images per class. This limitation is likely even more pronounced for the organoids dataset due to the higher variability and resolution of the images.

Further experimentation with more data could be required to reproduce the results seen with BBBC021.

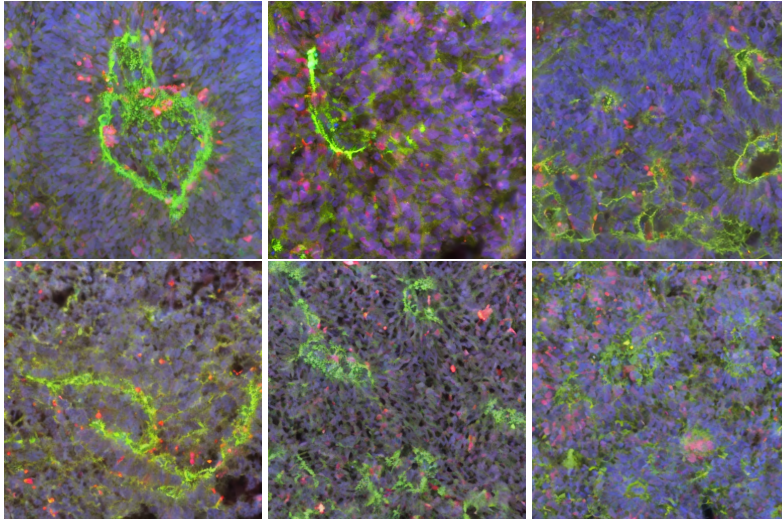


Figure 23: Examples of images generated by a model fine-tuned using LoRA on all 141 samples from the organoids dataset. The first row shows generated images for the *IMR* class, and the second row shows generated images for the *RSTS* class.

6 Conclusion

In this section, we will reflect on the most significant insights gained from our work and discuss potential avenues for further exploration.

Throughout this study, we explored the effectiveness of fine-tuning diffusion models to enhance their generative capabilities, particularly in the context of biomedical imaging. Our experiments focused on four fine-tuning methods: LoRA, SVDiff, attention fine-tuning, and full fine-tuning. We discovered that two of these methods, LoRA and SVDiff, are particularly effective at fine-tuning a latent diffusion model pre-trained on natural images to generalize well on a distribution of biological microscopy images. This is a significant finding because biological images are typically scarce and exhibit high variance, making them challenging to model. The ability to leverage the knowledge from a model trained on more readily available natural images is a powerful advantage. This approach could facilitate the development of advanced tools for scientific tasks, such as disease diagnosis or treatment planning, where data is often limited. Remarkably, we found that models could be effectively fine-tuned with as few as 800 images, making this approach particularly valuable in data-scarce environments such as biomedical research.

Moreover, LoRA and SVDiff methods update only a small fraction of the parameters compared to the vast number of parameters in diffusion models. This parameter efficiency allows these methods to adapt the models without requiring extensive computational resources, making them highly suitable for applications with limited data and computational power.

Beyond their ability to generalize across distributions, we also demonstrated that these fine-tuned models could effectively perform image-to-image translation using a conditional diffusion model. This capability is crucial for tasks where it is necessary to maintain high fidelity to the original image while altering specific features, such as simulating the effects of different treatments on cellular images.

However, our results were less satisfactory when applying these fine-tuning methods to datasets with higher resolution and greater variability between images, such as the organoids dataset. This outcome was somewhat expected due to the limited amount of data available, which may not have been sufficient for the models to generalize effectively. To fully understand how fine-tuning methods like LoRA and SVDiff perform on such complex datasets, additional experiments with larger datasets will be necessary.

Overall, the results suggest that fine-tuning pre-trained diffusion models with methods like LoRA and SVDiff can lead to significant improvements in generalization and applicability to complex biomedical imaging tasks. This approach not only maximizes the use of limited biological datasets but also opens new avenues for applying advanced deep learning techniques to critical areas of scientific research.

While the current study has yielded significant insights into the fine-tuning of diffusion models for biomedical imaging, there are several avenues for future research that could further enhance our understanding and application of these methods:

- **Exploring the Impact of Data Augmentation:** Investigating how different data augmentation techniques can influence the number of images required for models to generalize effectively. This could help determine whether augmentation can compensate for limited datasets.
- **Detailed image-to-image Quantitative Validation:** A more comprehensive quantitative evaluation of the image-to-image translation results could provide a deeper understanding of the fine-tuned models' performance.
- **Effectively Applying Fine-Tuning Methods to Complex Datasets:** Investigating the performance of fine-tuning methods on more complex datasets, where class differences are subtle, image resolution is higher, and variability between classes is significant, such as the organoids dataset. To accurately evaluate the effectiveness of methods like LoRA and SVDiff on such datasets, larger amounts of data would be required. However, such data is not currently available for the organoids dataset.
- **Determining the Generalization Thresholds for SVDiff and LoRA:** Future work could focus on identifying the exact number of images required for SVDiff and LoRA to reach the generalization regime. We estimated this threshold to be between 100 and 800 images, but precise data would help optimize the use of these methods, particularly in scenarios where data is very limited.
- **Exploring Layer-Specific Fine-Tuning:** Further research could explore the effects of applying fine-tuning methods to specific layers within the model. For instance, applying SVDiff and LoRA only to the attention layers, rather than across all layers, could provide insights into how different model components contribute to overall performance and may lead to more efficient fine-tuning strategies.

References

- [1] A. Bourou, T. Boyer, K. Daupin, V. Dubreuil, A. D. Thonel, V. Mezger, and A. Genovesio, “Phendiff: Revealing subtle phenotypes with diffusion models in real images,” 2024. Presented at MICCAI Workshop.
- [2] S. Velasco, A. J. Kedaigle, S. K. Simmons, A. Nash, M. Rocha, G. Quadrato, B. Paulsen, L. Nguyen, X. Adiconis, A. Regev, J. Z. Levin, and P. Arlotta, “Individual brain organoids reproducibly form cell diversity of the human cerebral cortex,” *Nature*, vol. 570, pp. 118–127, 2019.
- [3] A. de Thonel, J. K. Ahlskog, K. Daupin, V. Dubreuil, J. Berthelet, C. Chaput, G. Pires, C. Leonetti, R. Abane, L. Cordon Barris, I. Leray, A. L. Aalto, S. Naceri, M. Cordonnier, C. Benasolo, M. Sanial, A. Duchateau, A. Vihervaara, M. C. Puustinen, F. Miozzo, P. Fergelot, É. Lebigot, A. Verloes, P. Gressens, D. Lacombe, J. Gobbo, C. Garrido, S. D. Westerheide, L. David, M. Petitjean, O. Taboureau, F. Rodrigues-Lima, S. Passemard, D. Sabéran-Djoneidi, L. Nguyen, M. Lancaster, L. Sistonen, and V. Mezger, “Cbp-hsf2 structural and functional interplay in rubinstein-taybi neurodevelopmental disorder,” *Nat Commun*, vol. 13, no. 1, p. 7002, 2022.
- [4] A. Lamiabie, T. Champetier, F. Leonardi, E. Cohen, P. Sommer, D. Hardy, N. Argy, A. Massougbdji, E. Del Nery, G. Cottrell, Y.-J. Kwon, and A. Genovesio, “Revealing invisible cell phenotypes with conditional generative modeling,” *Nature Communications*, vol. 14, 2023.
- [5] H. Thanh-Tung, T. Tran, and S. Venkatesh, “On catastrophic forgetting and mode collapse in generative adversarial networks,” *International Joint Conference on Neural Networks*, pp. 1–10, 2018.
- [6] P. Dhariwal and A. Nichol, “Diffusion models beat gans on image synthesis,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 8780–8794, 2021.
- [7] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. Denton, S. K. S. Ghasemipour, B. K. Ayan, S. S. Mahdavi, R. G. Lopes, T. Salimans, J. Ho, D. J. Fleet, and M. Norouzi, “Photorealistic text-to-image diffusion models with deep language understanding,” *Advances in Neural Information Processing Systems*, vol. 35, p. 36479–36494, 2022.
- [8] S. Mo, M. Cho, and J. Shin, “Freeze discriminator: A simple baseline for fine-tuning gans,” 2020. In CVPRW.
- [9] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, “Finetuned language models are zero-shot learners,” 2021. In International Conference on Learning Representations.
- [10] D. Li and H. R. Zhang, “Improved regularization and robustness for fine-tuning in neural networks,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 27249–27262, 2021.
- [11] S. Huang, D. Xu, I. E. Yen, S. Chang, B. Li, S. Chen, M. Xie, H. Liu, and C. Ding, “Sparse progressive distillation: Resolving overfitting under pretrain-and-finetune paradigm,” *CoRR*, vol. abs/2110.08190, 2021.
- [12] E. Xie, L. Yao, H. Shi, Z. Liu, D. Zhou, Z. Liu, J. Li, and Z. Li, “Diffit: Unlocking transferability of large diffusion models via simple parameter-efficient fine-tuning,” *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4230–4239, 2023.
- [13] R. Dutt, L. Ericsson, P. Sanchez, S. A. Tsafaris, and T. Hospedales, “Parameter-efficient fine-tuning for medical image analysis: The missed opportunity,” 2024.
- [14] L. Han, Y. Li, H. Zhang, P. Milanfar, D. Metaxas, and F. Yang, “Svdif: Compact parameter space for diffusion fine-tuning,” *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 7323–7334, 2023.
- [15] P. Caie, R. Walls, A. Ingleston-Orme, S. Daya, T. Houslay, R. Eagle, M. Roberts, and N. Carragher, “High-content phenotypic profiling of drug response signatures across distinct cancer cells,” *Molecular cancer therapeutics*, vol. 9, pp. 1913–26, 06 2010.

- [16] G. Antolini and M. Colizzi, “Where do neurodevelopmental disorders go? casting the eye away from childhood towards adulthood,” *Healthcare*, vol. 11, p. 1015, 04 2023.
- [17] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [18] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” In International Conference on Machine Learning.
- [19] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” 2022. In International Conference on Learning Representations.
- [20] D. P. Kingma and M. Welling, “An introduction to variational autoencoders,” *Foundations and Trends® in Machine Learning*, vol. 12, no. 4, p. 307–392, 2019.
- [21] S. H. Chan, “Tutorial on diffusion models for imaging and vision,” 2024. arXiv:2403.18103.
- [22] C. Luo, “Understanding diffusion models: A unified perspective,” 2022. arXiv:2208.11970.
- [23] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” 2020. International Conference on Learning Representations.
- [24] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” 2021. In International Conference on Learning Representations.
- [25] B. D. O. Anderson, “Reverse-time diffusion equation models,” *Stochastic Processes and their Applications*, vol. 12(3), p. 313–326, 1982.
- [26] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10684–10695, 2021.
- [27] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” *Medical image computing and computer-assisted intervention, proceedings, part III 18*, pp. 234–241, 2015.
- [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [29] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna, and R. Rombach, “Sdxl: Improving latent diffusion models for high-resolution image synthesis,” 2023. arXiv:2307.01952.
- [30] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” 2021. In IEEE Automatic Speech Recognition and Understanding Workshop (ASRU).
- [31] H. Cai, C. Gan, L. Zhu, and S. Han, “Tiny transfer learning: Towards memory-efficient on-device learning,” *CoRR*, vol. abs/2007.11622, 2020.
- [32] S. Basu, D. Massiceti, S. X. Hu, and S. Feizi, “Strong baselines for parameter efficient few-shot fine-tuning,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38-10, pp. 11024–11031, 2023.
- [33] J. Frankle, D. J. Schwab, and A. S. Morcos, “Training batchnorm and only batchnorm: On the expressive power of random features in cnns,” 2020. In International Conference on Learning Representations.
- [34] E. B. Zaken, S. Ravfogel, and Y. Goldberg, “Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models,” *CoRR*, vol. abs/2106.10199, 2021.
- [35] P. Tschandl, C. Rosendahl, and H. Kittler, “The HAM10000 dataset: A large collection of multi-source dermatoscopic images of common pigmented skin lesions,” *Scientific Data*, vol. 5-180161, 2018.

- [36] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” *CoRR*, vol. abs/2010.11929, 2020.
- [37] A. Aghajanyan, L. Zettlemoyer, and S. Gupta, “Intrinsic dimensionality explains the effectiveness of language model fine-tuning,” *CoRR*, vol. abs/2012.13255, 2020.
- [38] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan, “Peft: State-of-the-art parameter-efficient fine-tuning methods.” <https://github.com/huggingface/peft>, 2022.
- [39] E. Robb, W. Chu, A. Kumar, and J. Huang, “Few-shot adaptation of generative adversarial networks,” *CoRR*, vol. abs/2010.11943, 2020.
- [40] S. Brunton and J. Kutz, *Data-Driven Science and Engineering*, pp. 3–46. Cambridge University Press, 02 2019.
- [41] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, “Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation,” 2023. In Conference on Computer Vision and Pattern Recognition (CVPR).
- [42] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, “Hierarchical text-conditional image generation with clip latents,” 2022. arXiv:2005.14165.
- [43] X. Su, J. Song, C. Meng, and S. Ermon, “Dual diffusion implicit bridges for image-to-image translation,” 2023. In International Conference on Learning Representations.
- [44] S. Jayasumana, S. Ramalingam, A. Veit, D. Glasner, A. Chakrabarti, and S. Kumar, “Rethinking fid: Towards a better evaluation metric for image generation,” 2024. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- [45] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, G. Klambauer, and S. Hochreiter, “Gans trained by a two time-scale update rule converge to a nash equilibrium,” *Advances in Neural Information Processing Systems*, pp. 6626–6637, 2017.
- [46] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2818–2826, 2015.
- [47] E. Nunn, P. Khadivi, and S. Samavi, “Compound frechet inception distance for quality assessment of gan created images,” 2021. arXiv:2106.08575.
- [48] G. Somepalli, V. Singla, M. Goldblum, J. Geiping, and T. Goldstein, “Diffusion art or digital forgery? investigating data replication in diffusion models,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6048–6058, 2022.
- [49] Z. Kadkhodaie, F. Guth, E. P. Simoncelli, and S. Mallat, “Generalization in diffusion models arises from geometry-adaptive harmonic representations,” 2024. In International Conference on Learning Representations.

A Appendix

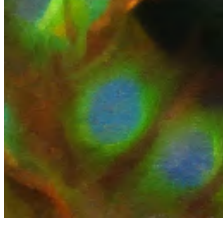
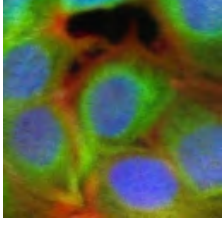
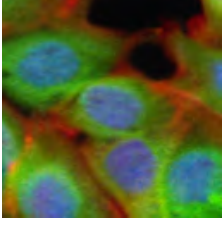
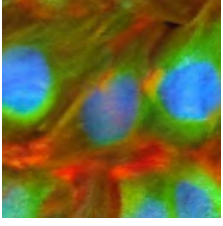
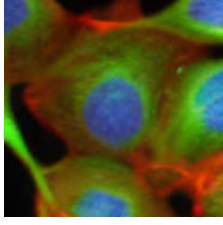
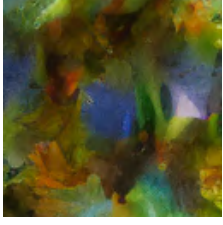
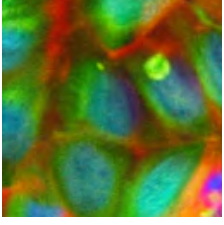
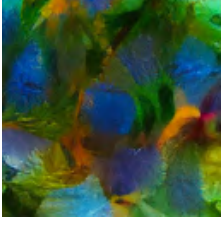
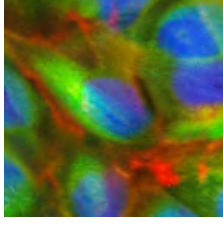
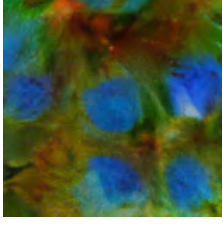
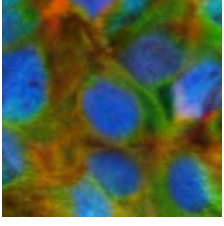
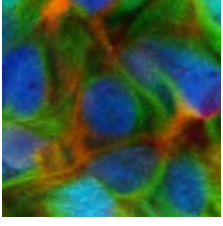
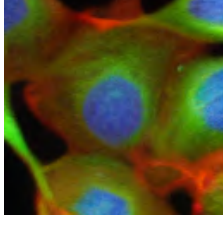
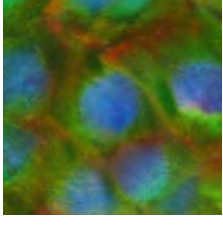
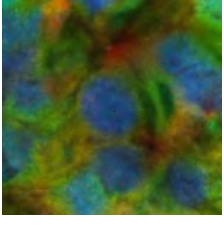
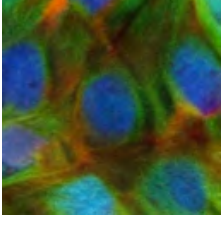
Method	N = 10	N = 50	N = 100	N = 1000
Full fine-tuning				
Attention fine-tuning				
LoRA				
SVDiff				

Table 3: Examples of images of the class *DMSO* generated using four different methods, each trained with four different training sizes per class. All images were generated with the same Gaussian noise.

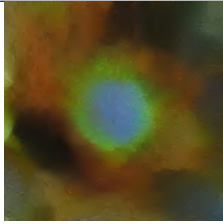
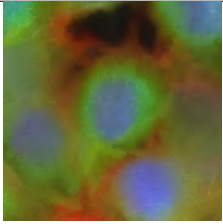
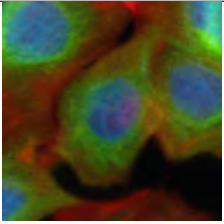
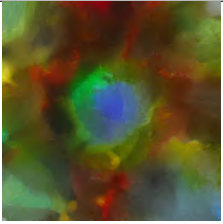
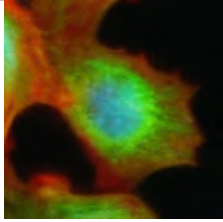
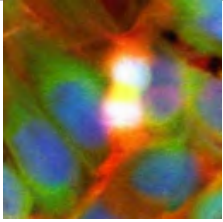
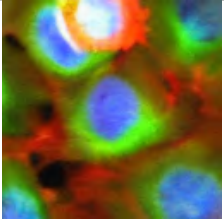
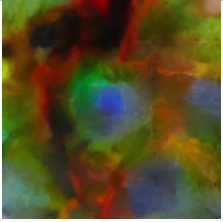
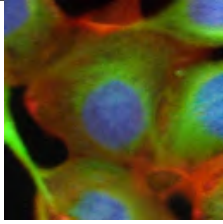
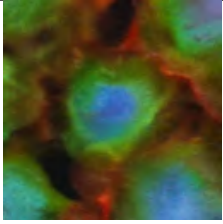
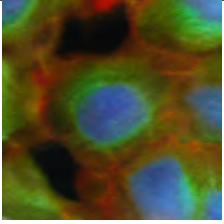
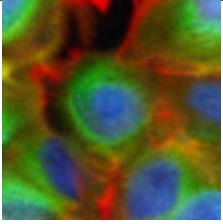
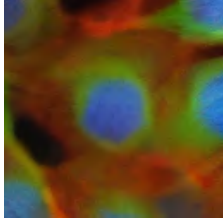
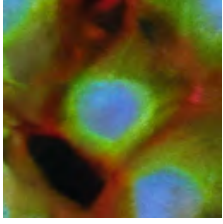
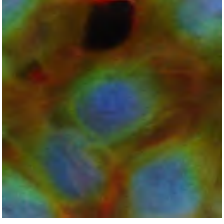
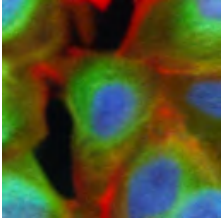
Method	N = 10	N = 50	N = 100	N = 1000
Full fine-tuning				
Attention fine-tuning				
LoRA				
SVDiff				

Table 4: More examples of images of the class *DMSO* generated using four different methods, each trained with four different training sizes per class. All images were generated with the same Gaussian noise.

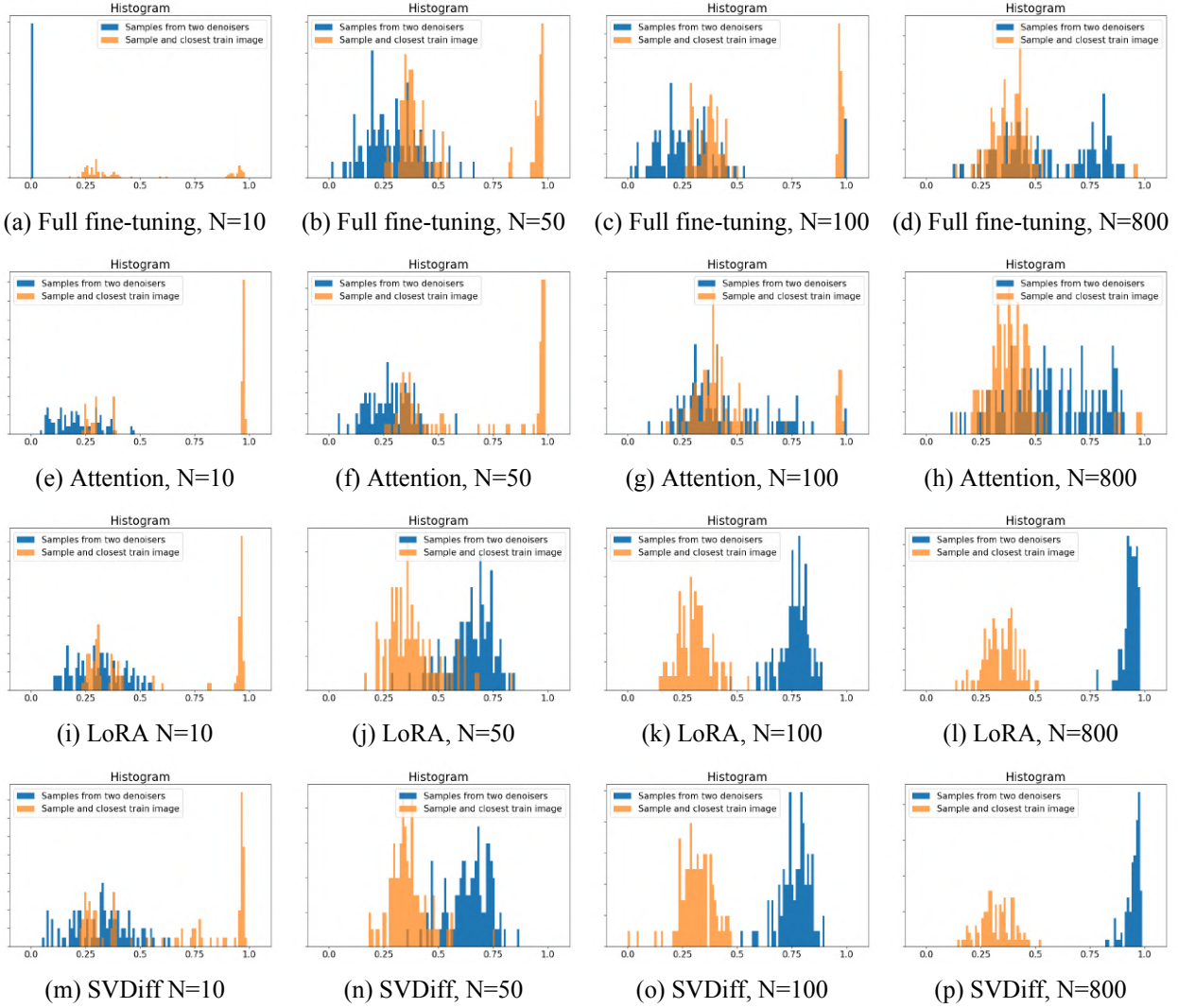


Figure 24: Generalization histograms for class *DMSO* as described in Section 4.3. The orange bars represent the correlations between images generated by the same seed from two models trained on different non-overlapping datasets. The blue bars represent the correlations between the generated images and their closest images in the training dataset.

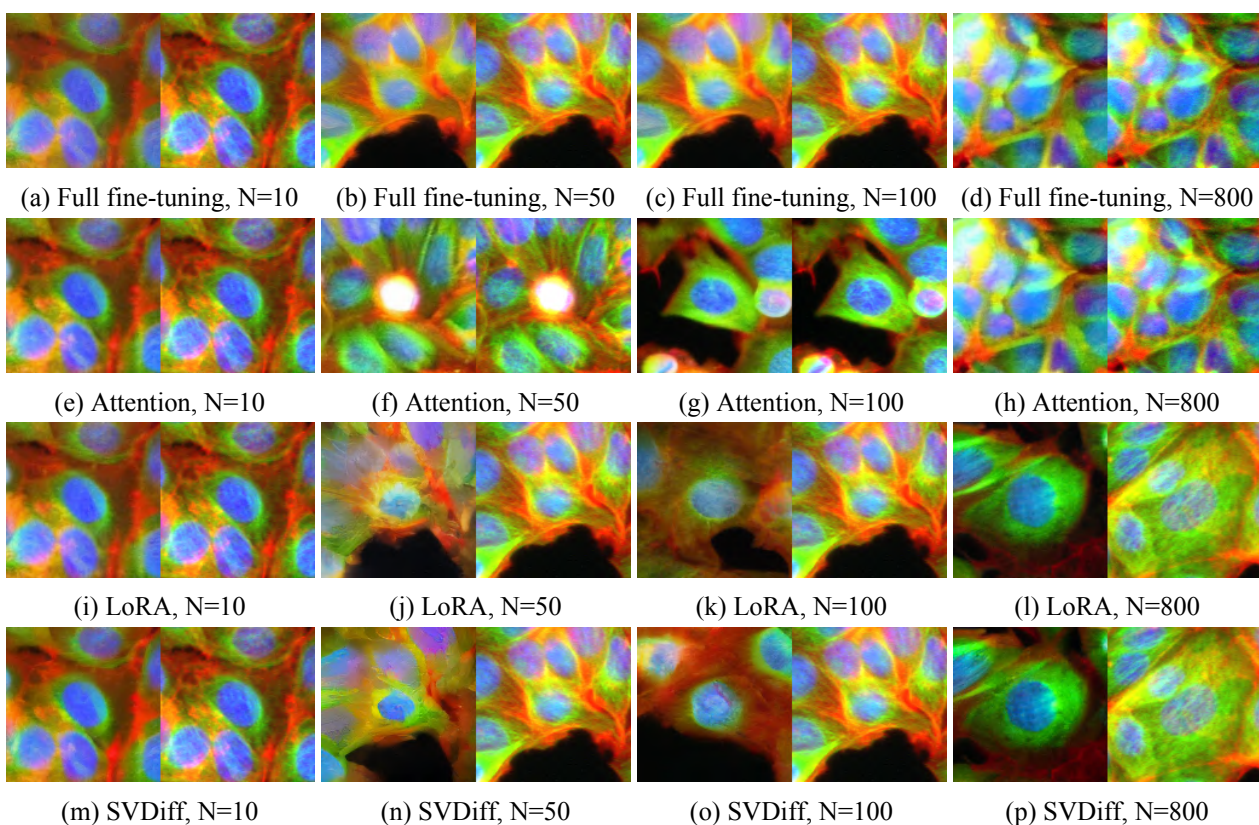


Figure 25: Top-1 *DMSO* most similar generated and training image for each model, according to cosine similarity distance. For each pair, the generated image is on the left and the closest training image is on the right.